Indexing and Trees (Chapter 17)

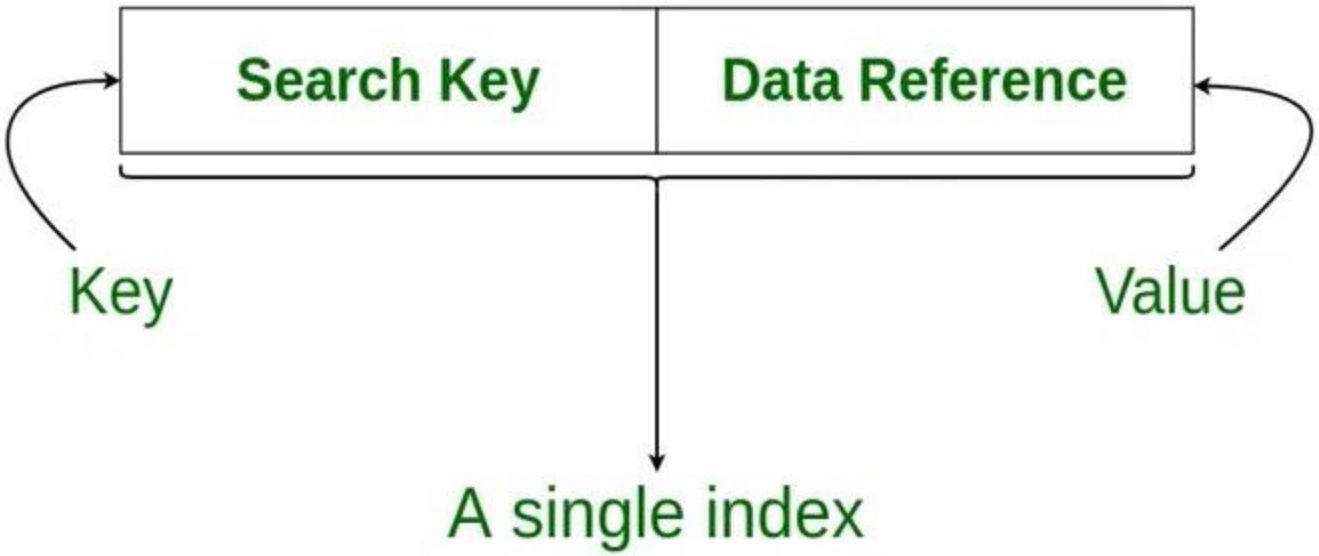
Introduction

- Indexing improves database performance by minimizing the number of disc visits required to fulfill a query.
- It is a data structure technique used to locate and quickly access data in databases.

Introduction

- The main key or candidate key of the table is duplicated in the first column, which is the Search key.
- To speed up data retrieval, the values are also kept in sorted order. It should be highlighted that sorting the data is not required.
- The second column is the Data Reference or Pointer which contains a set of pointers holding the address of the disk block where that particular key value can be found.

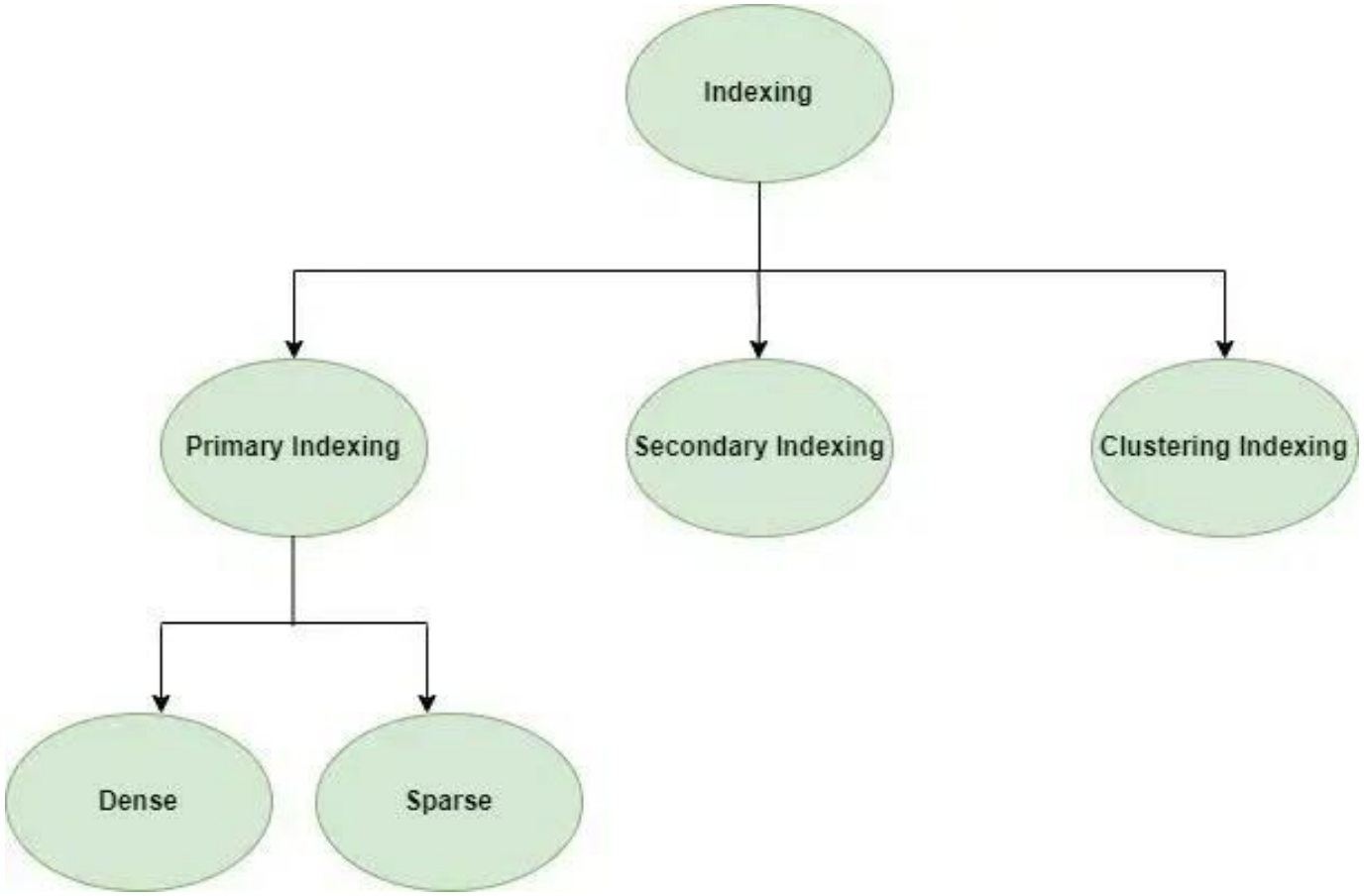
Structure of an Index in Database



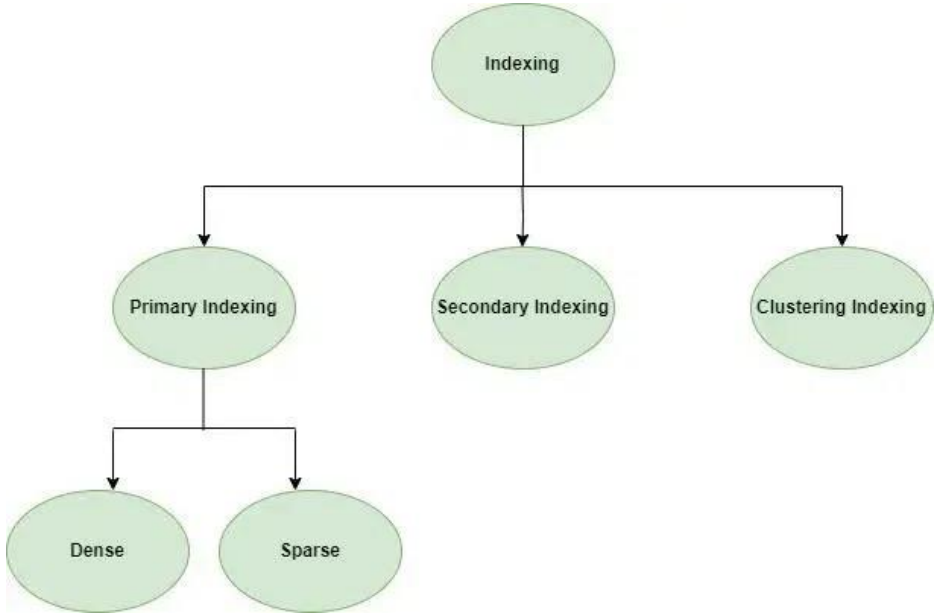
Attributes of Indexing

- **Access Types:** This refers to the type of access such as value-based search, range access, etc.
- **Access Time:** It refers to the time needed to find a particular data element or set of elements.
- **Insertion Time:** It refers to the time taken to find the appropriate space and insert new data.
- **Deletion Time:** Time taken to find an item and delete it as well as update the index structure.
- **Space Overhead:** It refers to the additional space required by the index.

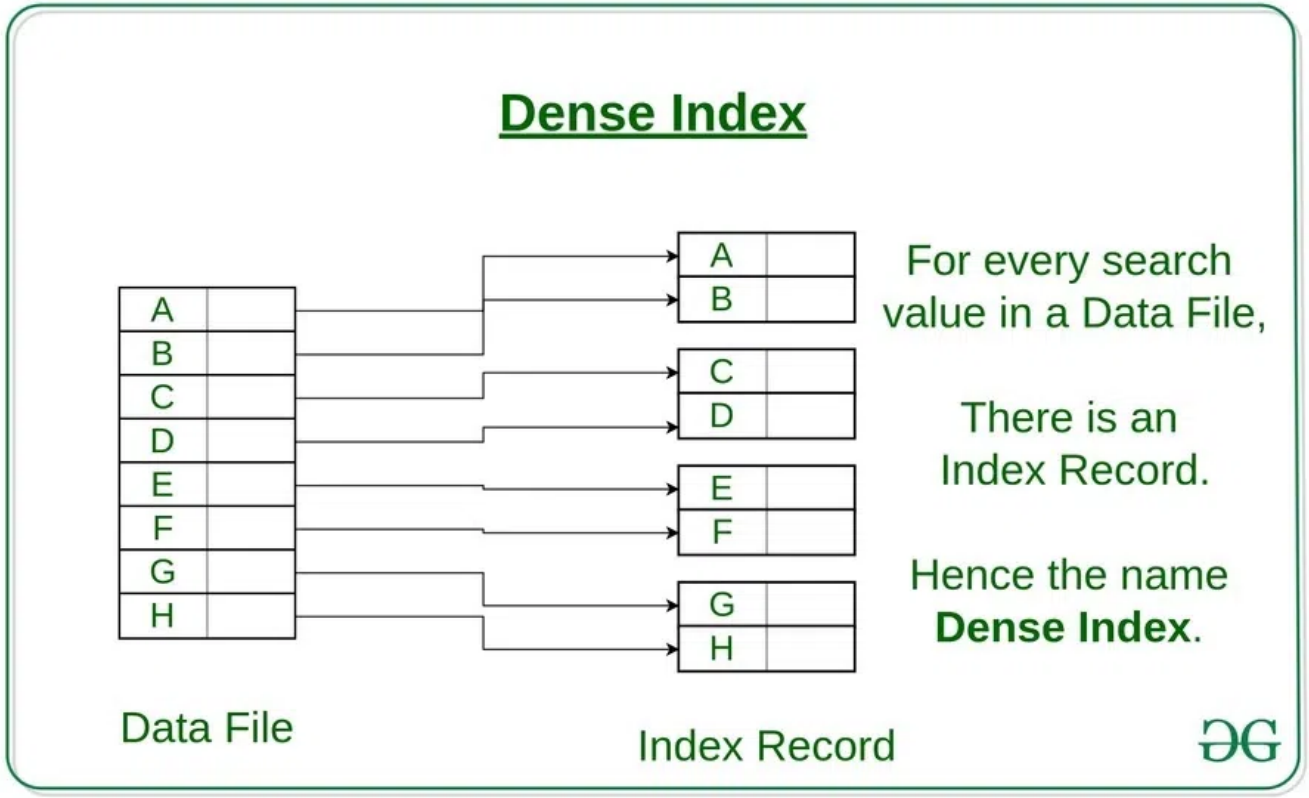
Types of indexing: Single level indexing



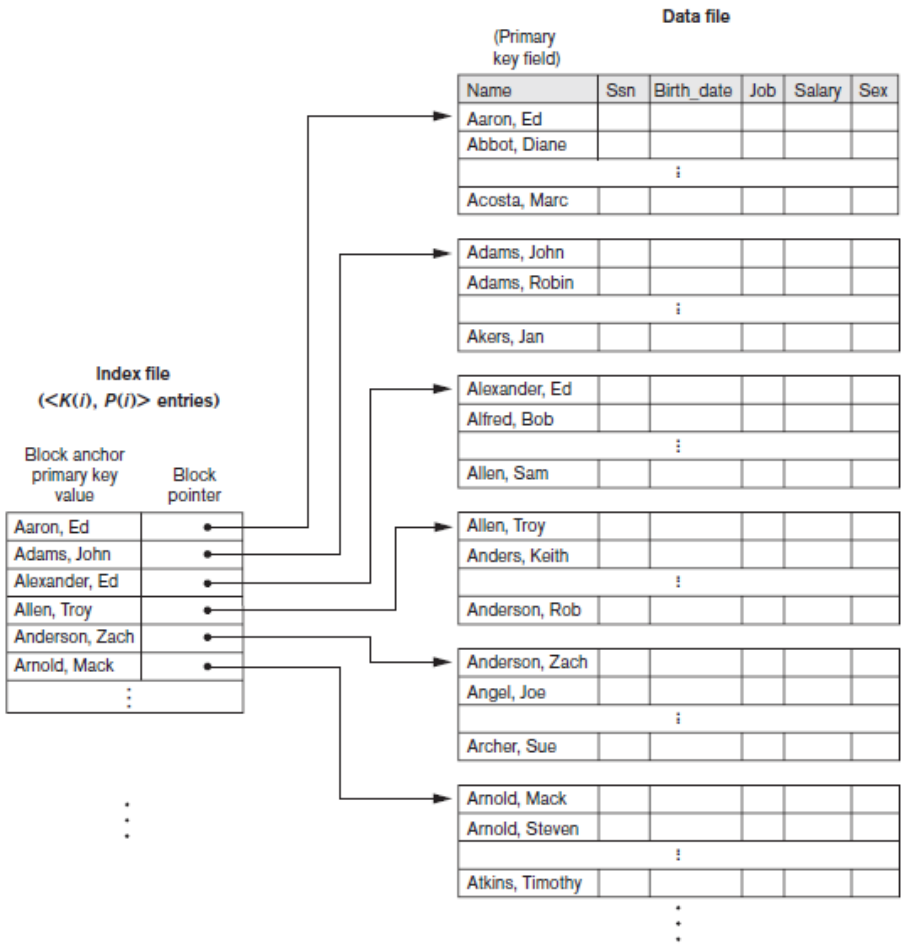
Primary Indexing: This type of indexing is usually associated with the primary key of a database table. It is the main index that is used to quickly locate records.



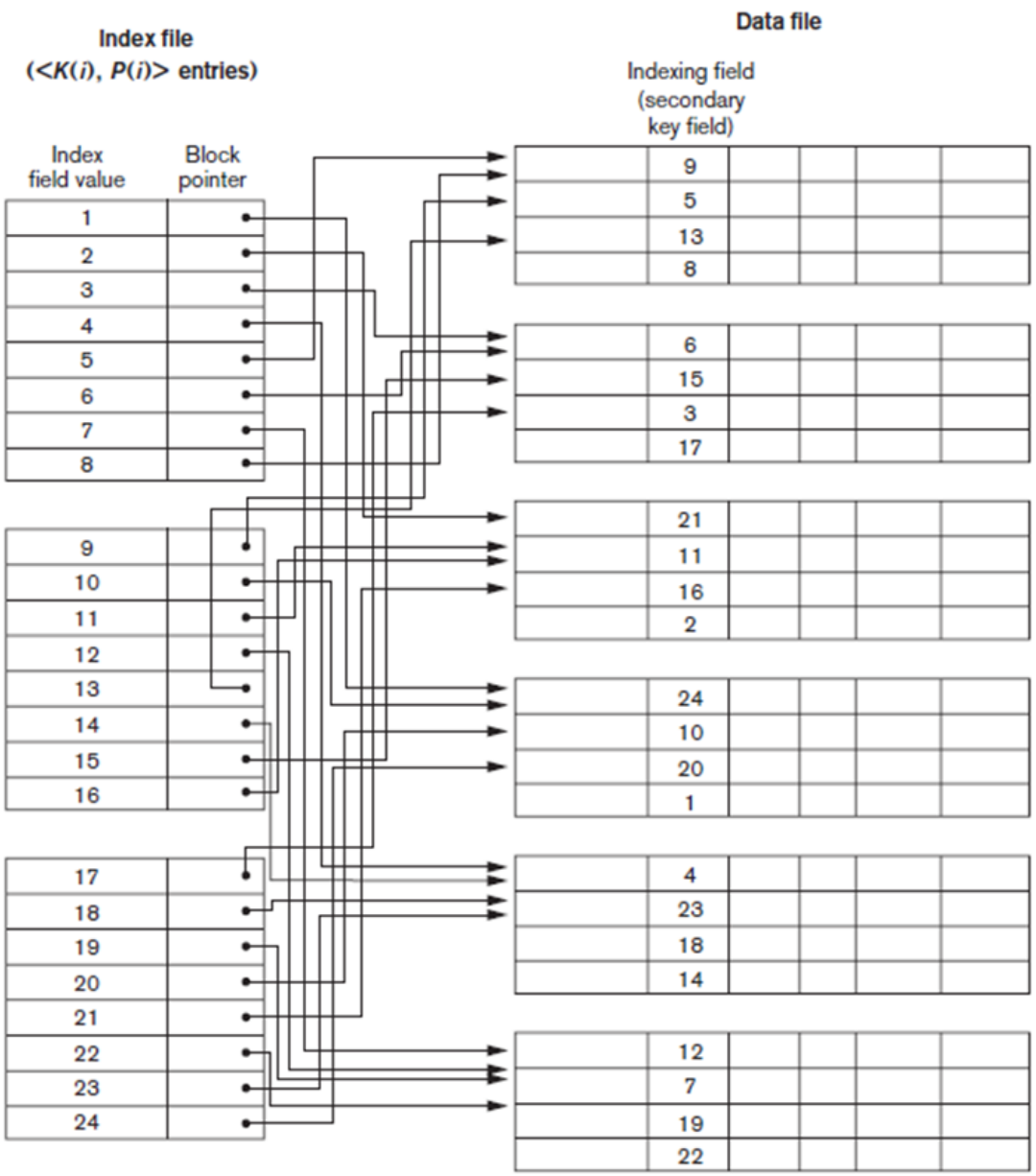
Dense Index: A dense index has an index entry for every single record in the database. This means that the index maps every record with a key in the database.



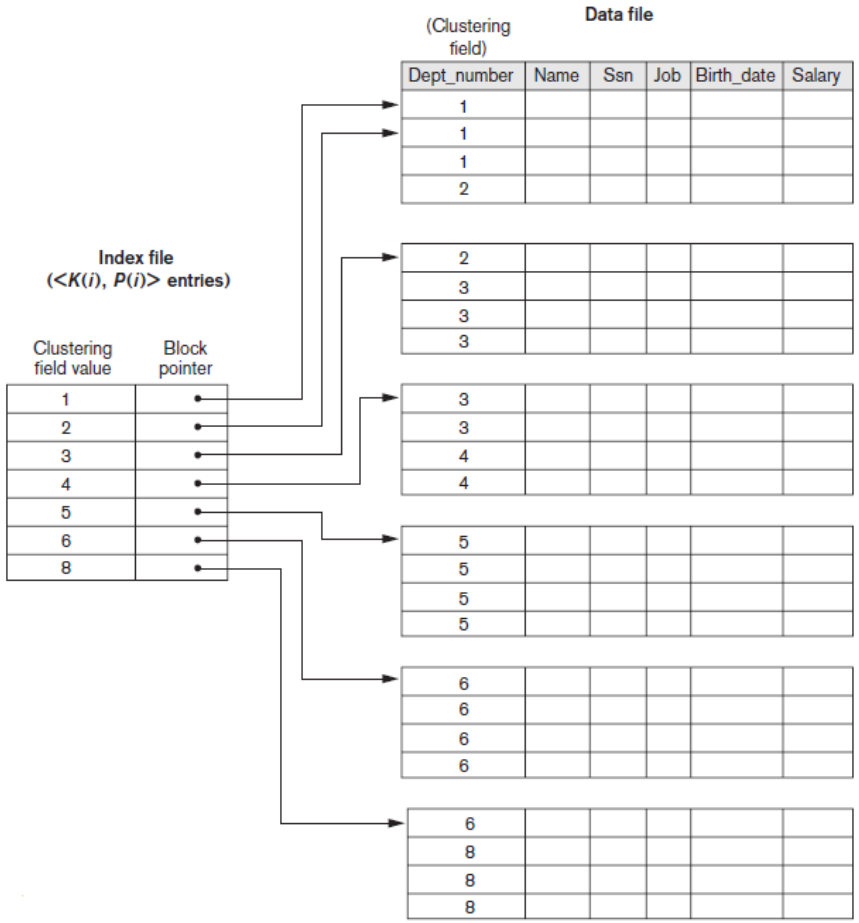
Sparse Index: In contrast, a sparse index does not include all the keys in the dataset. Instead, it includes pointers to blocks of records, and the keys in the index are spaced at intervals. This type of index requires less space than a dense index.



Secondary Indexing: This type of index is used to access fields that are not part of the primary key. It is used to improve the speed of data retrieval for columns other than the primary key.



Clustering Indexing: This type of indexing is used when records are physically stored on the disk in a cluster based on one or more columns, which are not necessarily the primary key. The index will point to these clusters.



Summary

	Index Field Used for Physical Ordering of the File	Index Field Not Used for Physical Ordering of the File
Indexing field is key	Primary index	Secondary index (Key)
Indexing field is nonkey	Clustering index	Secondary index (NonKey)

Type of Index	Number of (First-Level) Index Entries	Dense or Nondense (Sparse)	Block Anchoring on the Data File
Primary	Number of blocks in data file	Nondense	Yes
Clustering	Number of distinct index field values	Nondense	Yes/no[a]
Secondary (key)	Number of records in data file	Dense	No
Secondary (nonkey)	Number of records[b] or number of distinct index field values[c]	Dense or Nondense	No

[a]Yes if every distinct value of the ordering field starts a new block; no otherwise.
[b]For option 1.
[c]For options 2 and 3.

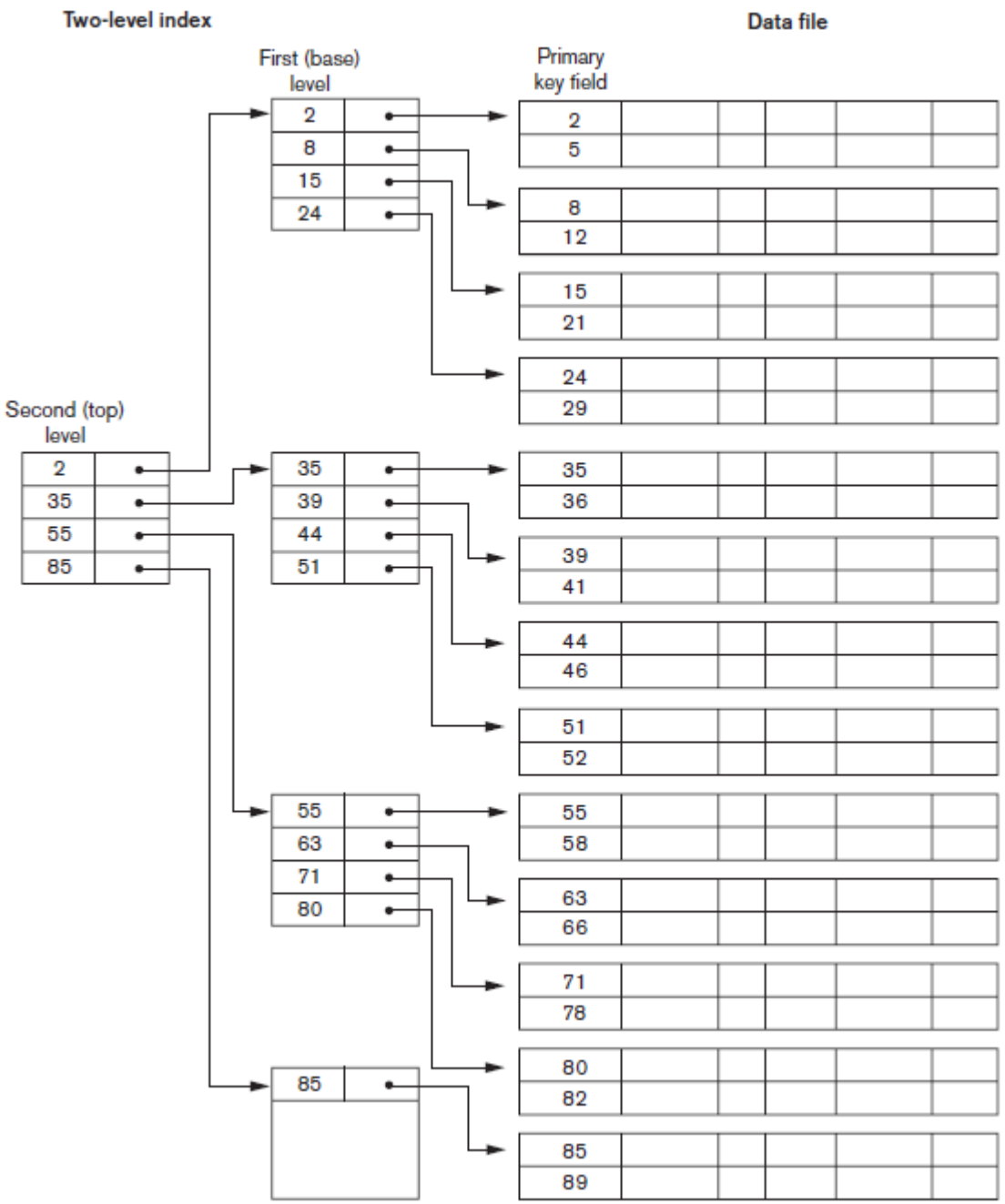
Multi-level indexing

- Multi-level indexing is used in databases to improve the efficiency of data retrieval, especially when dealing with large databases.

- The purpose is to further minimize the search space when looking for a record. Instead of scanning an entire index, a search first goes through a top level.

- Reduces the number of disk accesses required to find a record, which can significantly improve performance in large databases.

Figure 17.6 A two-level primary index

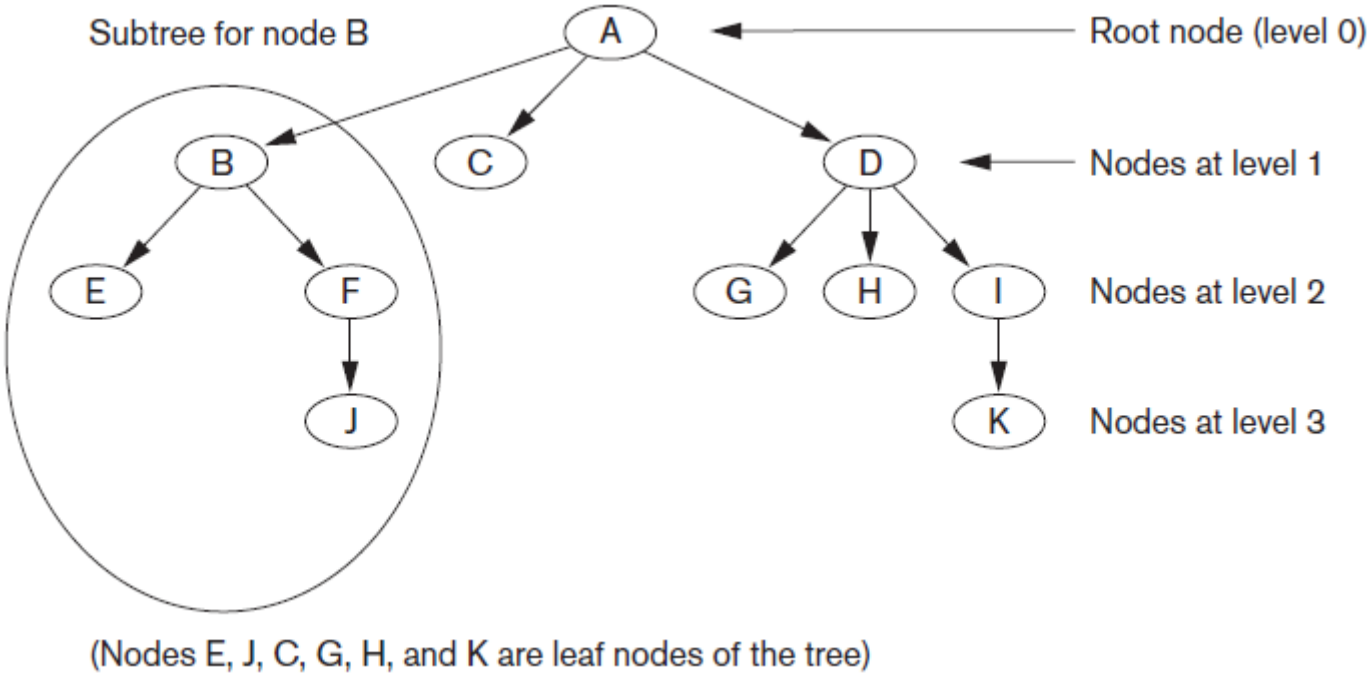


Static Multilevel Indexes Using ISAM Trees

Tree data structure terminology

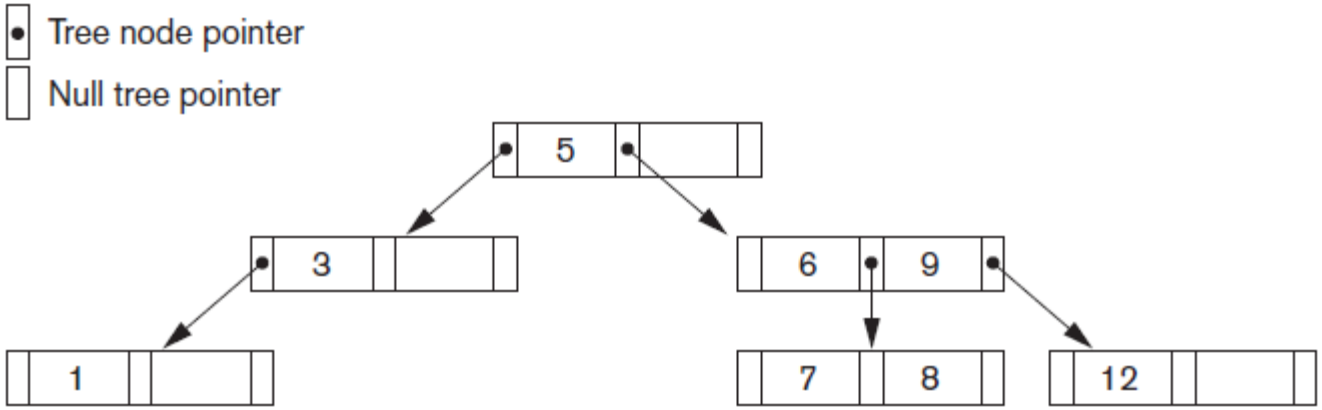
- Tree data structure terminology
  - Tree is formed of nodes
  - Each node (except root) has one parent and zero or more child nodes
  - Leaf node has no child nodes
    - Unbalanced if leaf nodes occur at different levels
  - Nonleaf node called internal node
  - Subtree of node consists of node and all descendant nodes

Tree Data Structure



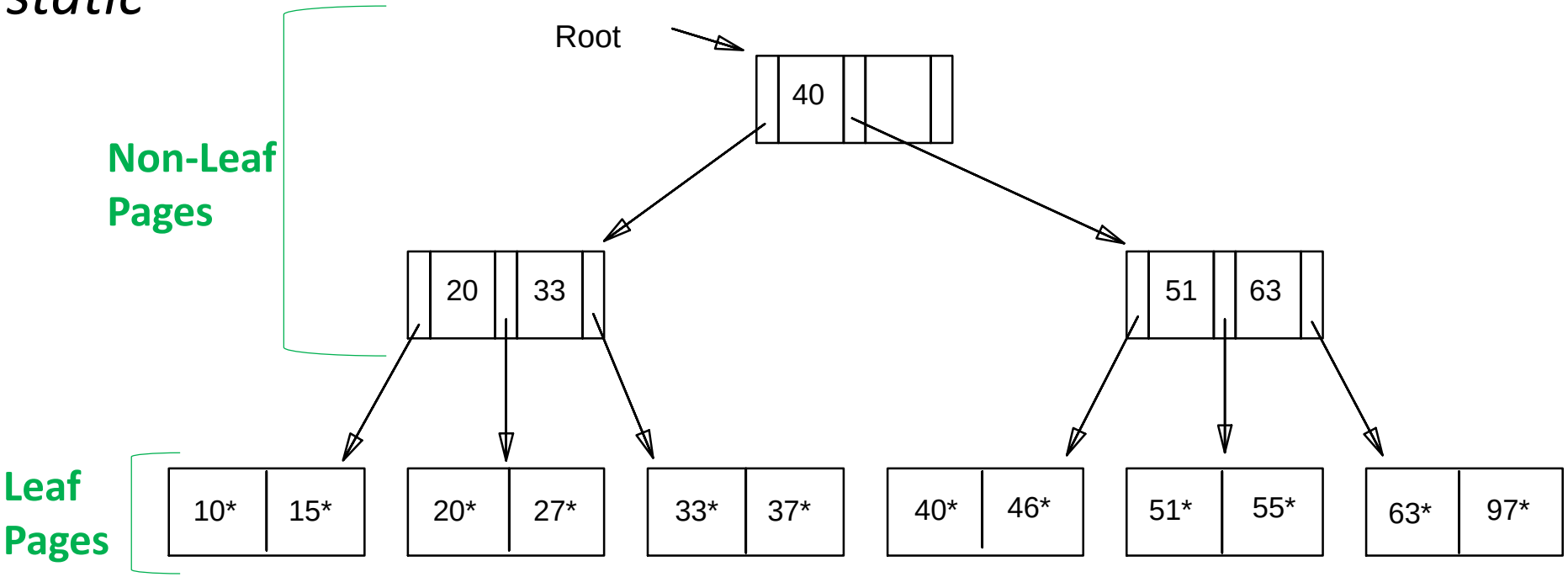
Search tree

- Algorithms necessary for inserting and deleting search values into and from the tree



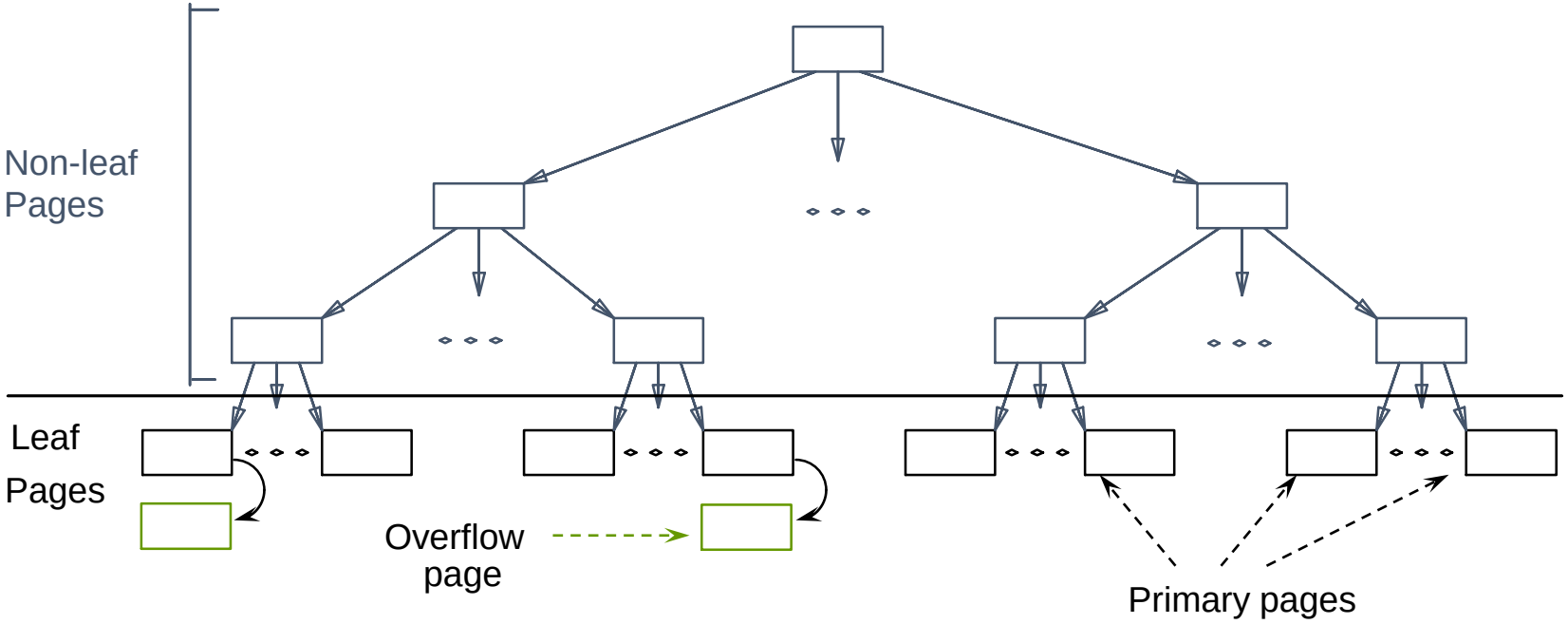
ISAM Trees

- Indexed Sequential Access Method (ISAM) trees are *static*



ISAM Trees: Page Overflows

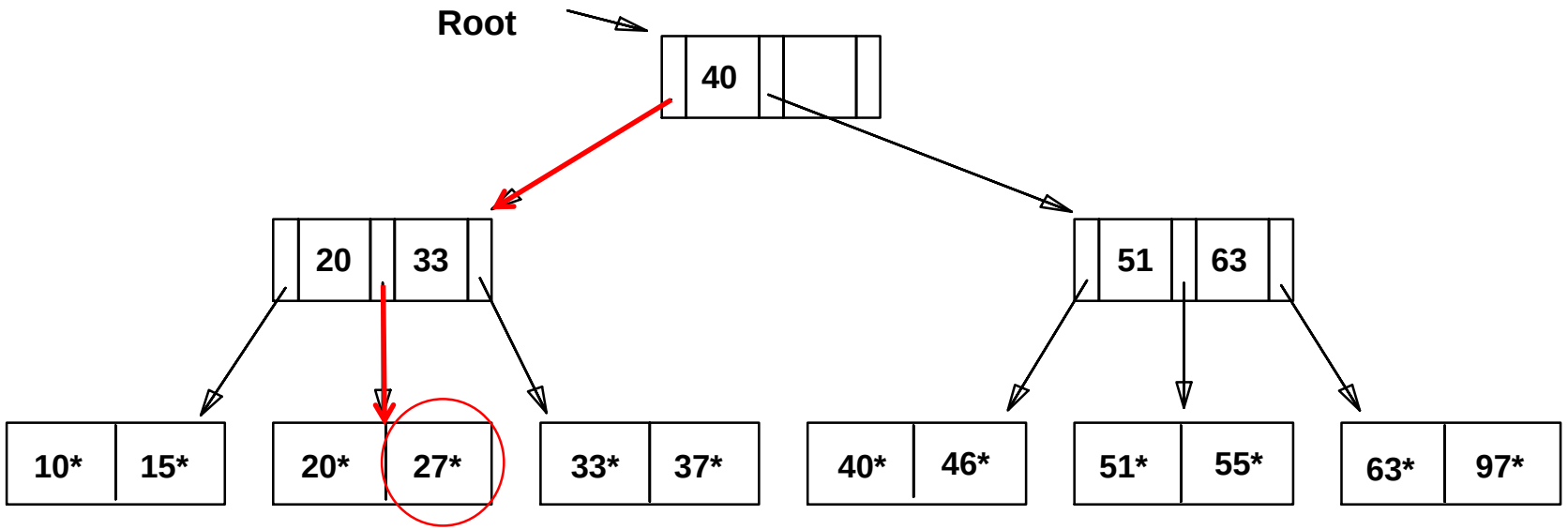
- What if there are a lot of insertions after creating the tree?



ISAM: Searching for Entries

- Search begins at root, and key comparisons direct it to a leaf

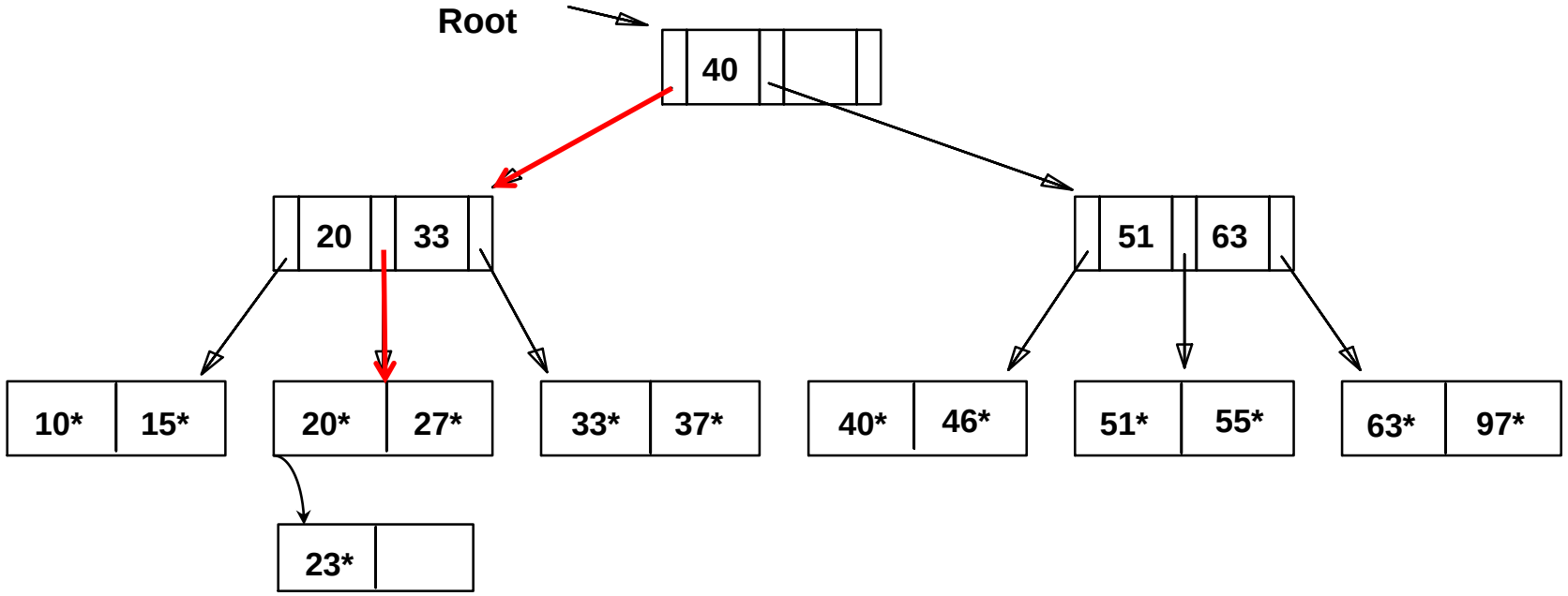
- Search for 27*



ISAM: Inserting Entries

- The appropriate page is determined as for a search, and the entry is inserted (with overflow pages added if necessary)

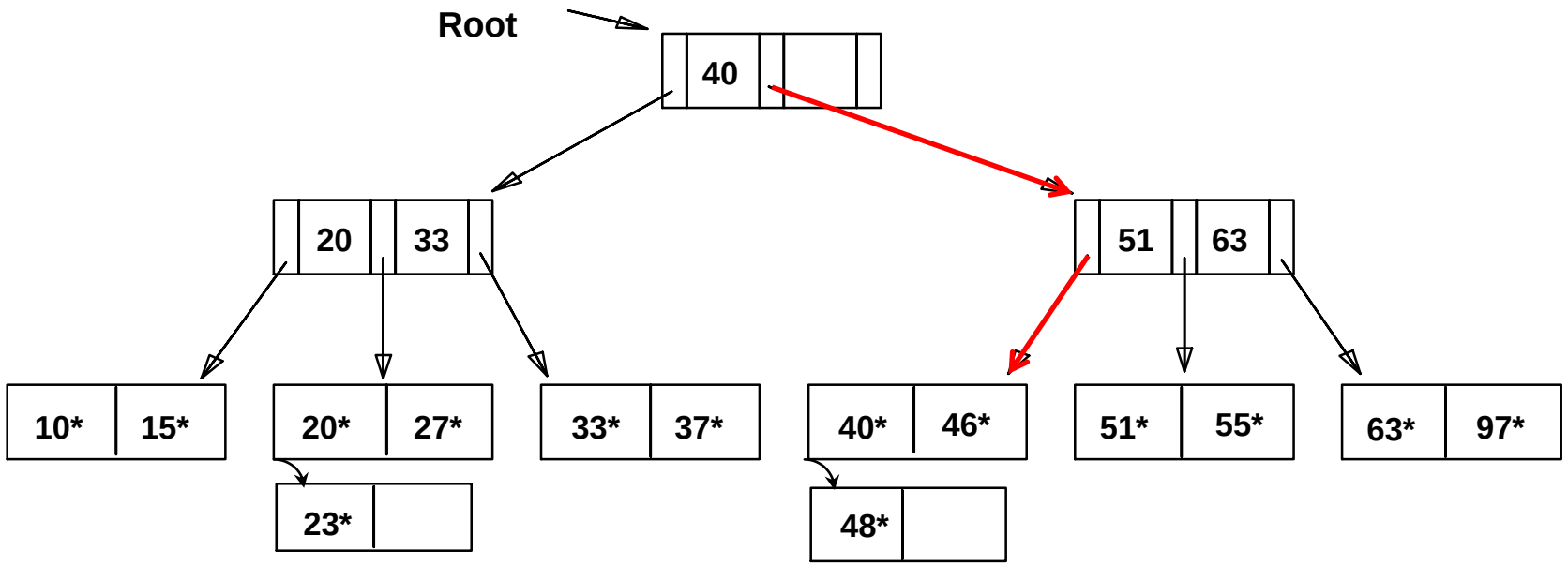
- Insert 23*



ISAM: Inserting Entries

- The appropriate page is determined as for a search, and the entry is inserted (with overflow pages added if necessary)

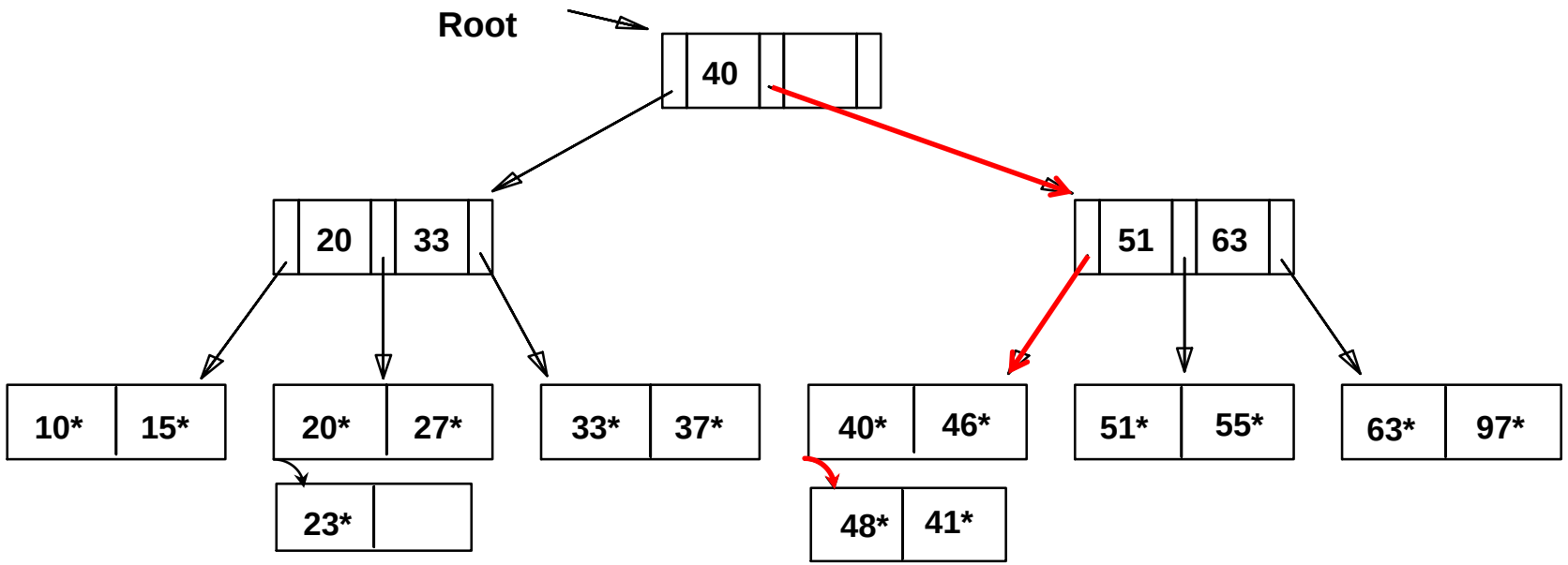
- Insert 48*



ISAM: Inserting Entries

- The appropriate page is determined as for a search, and the entry is inserted (with overflow pages added if necessary)

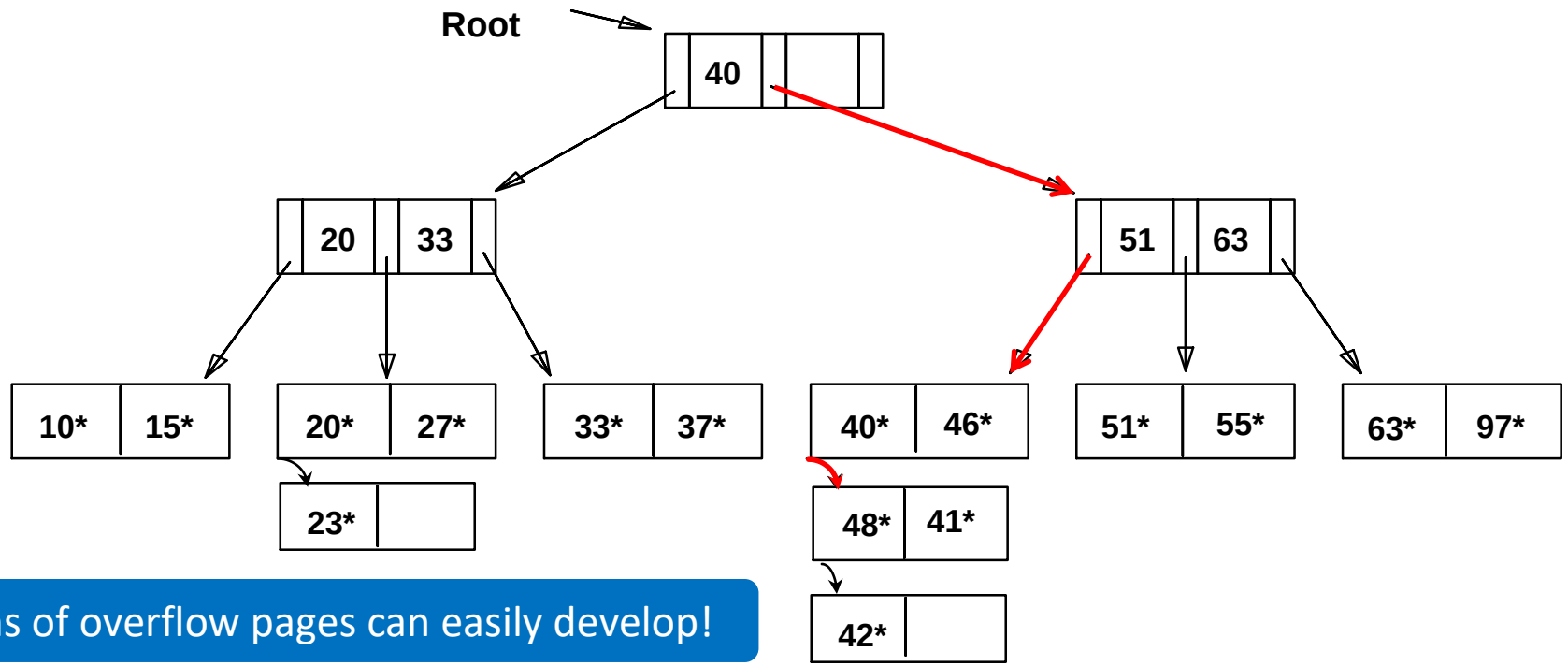
- Insert 41*



ISAM: Inserting Entries

- The appropriate page is determined as for a search, and the entry is inserted (with overflow pages added if necessary)

- Insert 42*

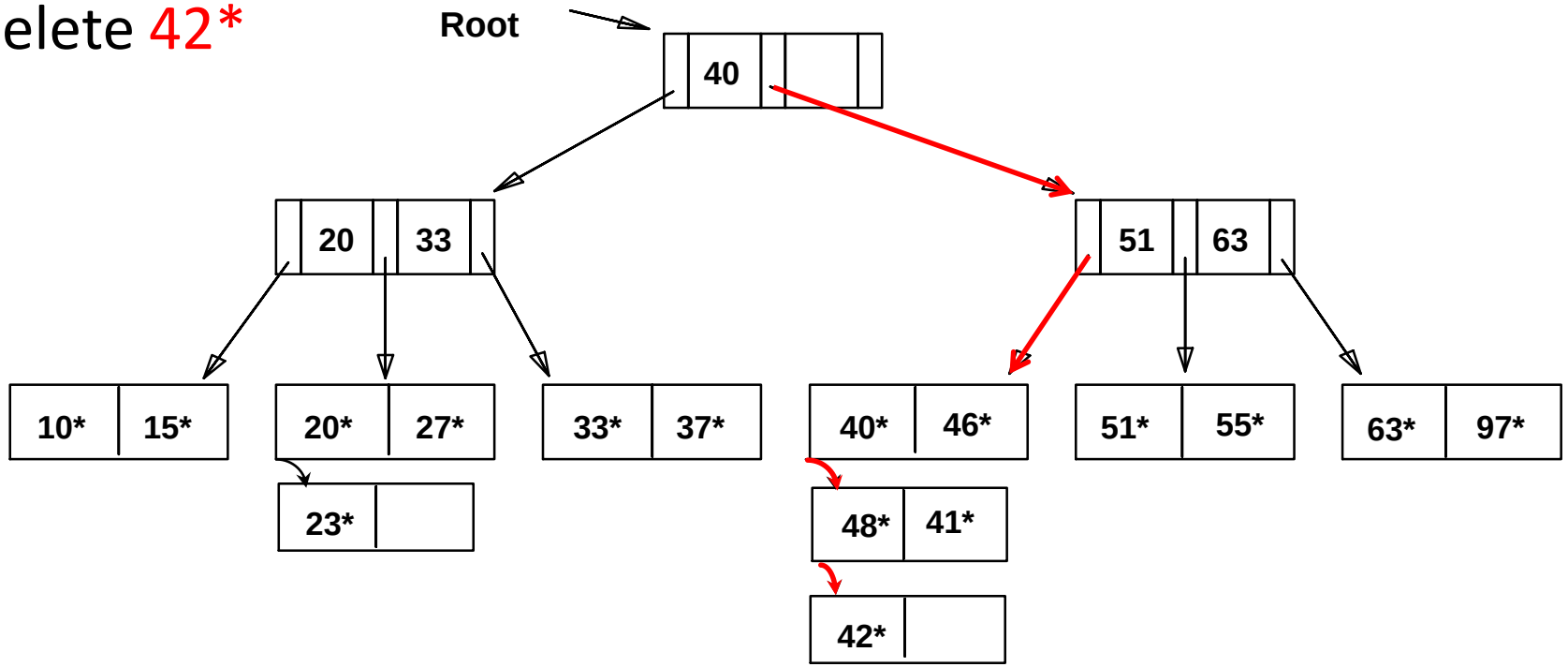


Chains of overflow pages can easily develop!

ISAM: Deleting Entries

- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

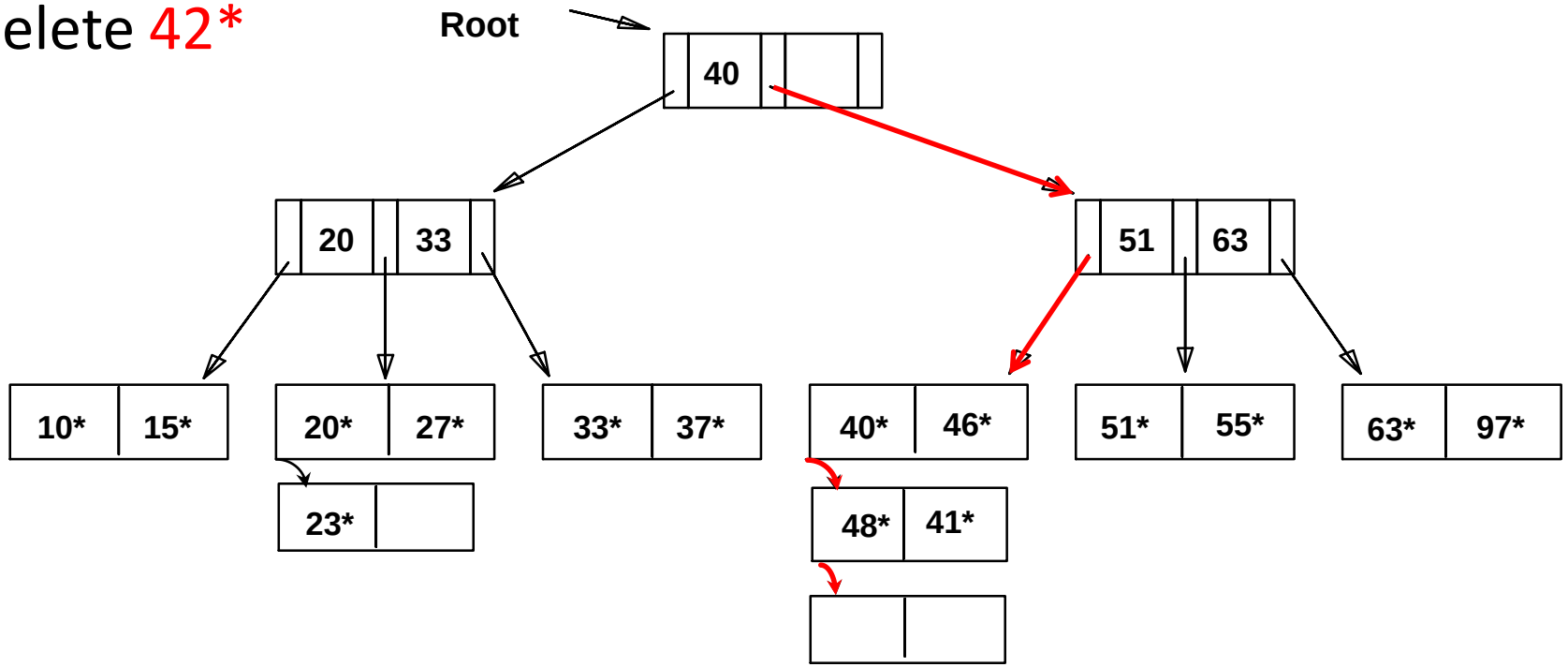
- Delete 42*



ISAM: Deleting Entries

- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

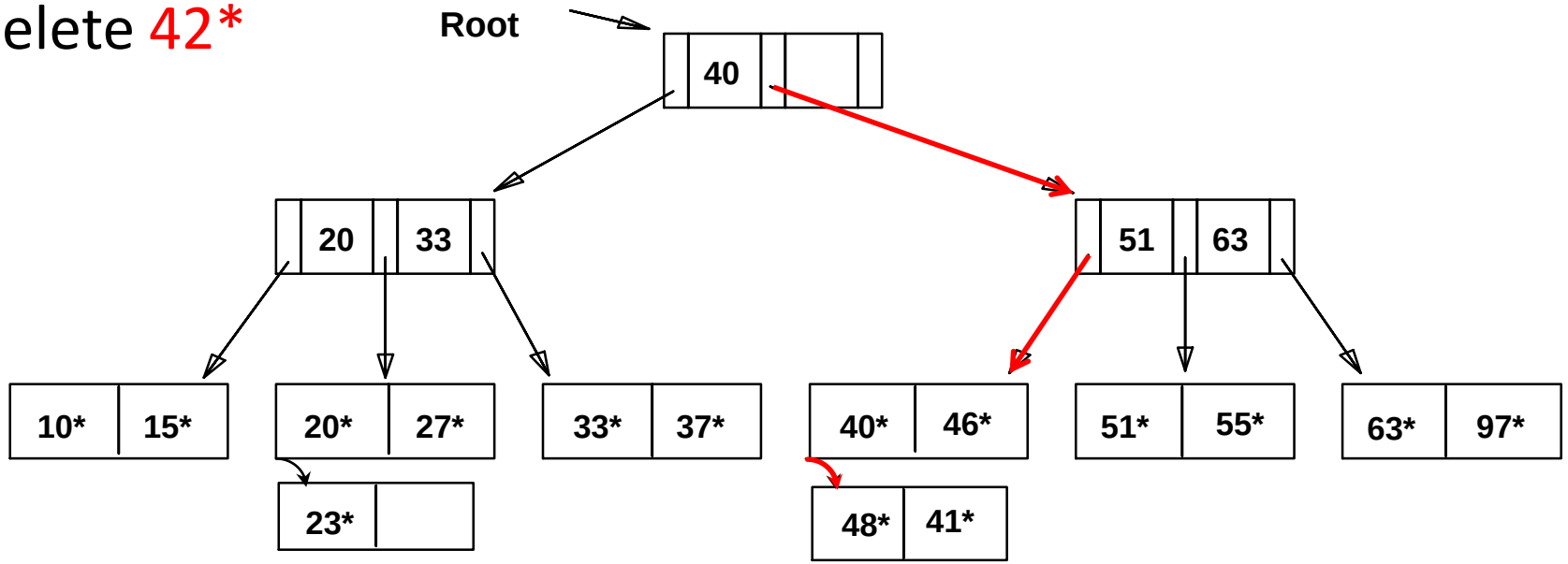
- Delete 42*



ISAM: Deleting Entries

- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

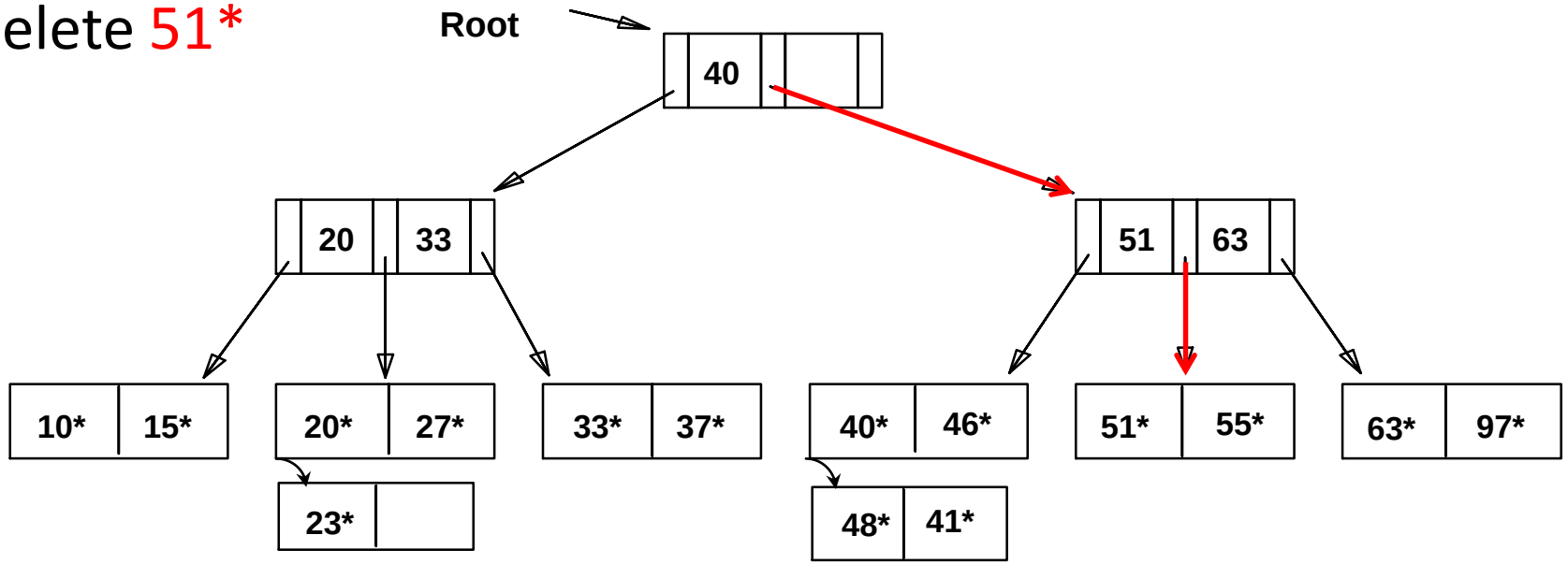
- Delete 42*



ISAM: Deleting Entries

- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

- Delete 51*

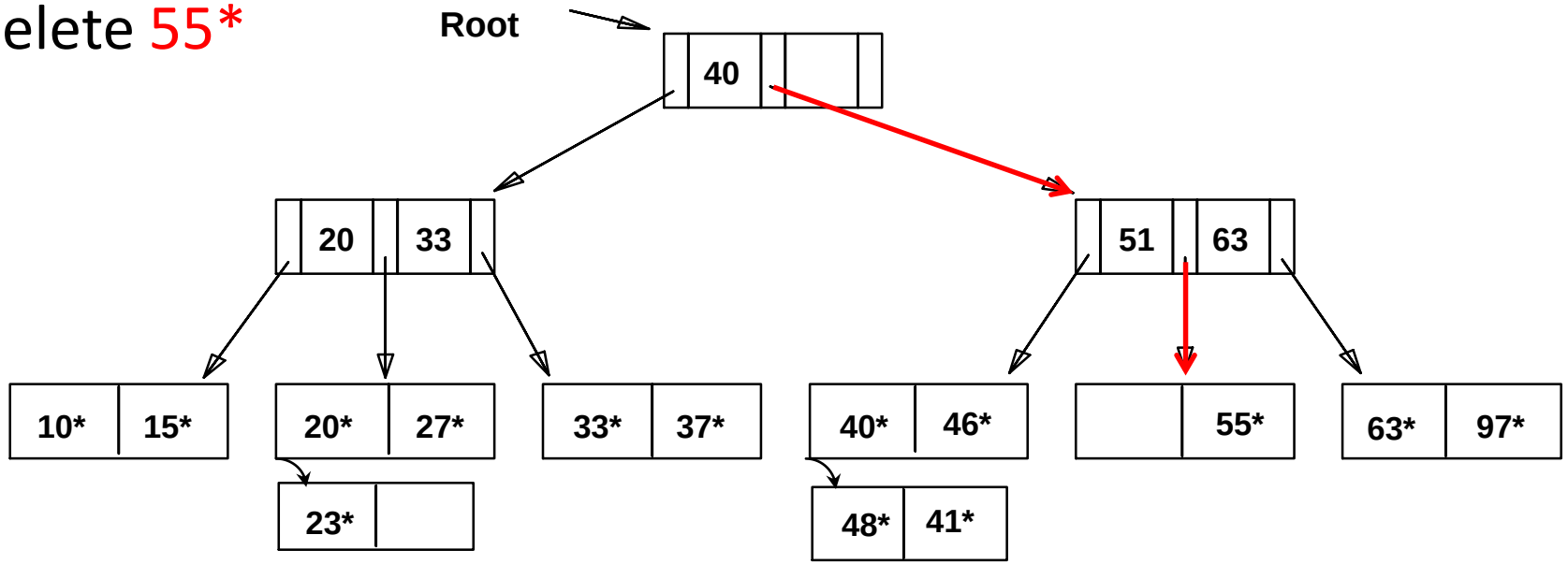


Note that 51 still appears in an index entry, but not in the leaf!

ISAM: Deleting Entries

- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

- Delete 55*



Primary pages are NOT removed, even if they become empty!

ISAM: Some Issues

- Once an ISAM file is created, insertions and deletions affect only the contents of leaf pages (i.e., *ISAM is a static structure*!)
- Since index-level pages are *never* modified, there is no need to *lock* them during insertions/deletions
  - Critical for concurrency!
- Long overflow chains can develop easily
- If the data distribution and size are relatively static, ISAM might be a good choice to pursue!

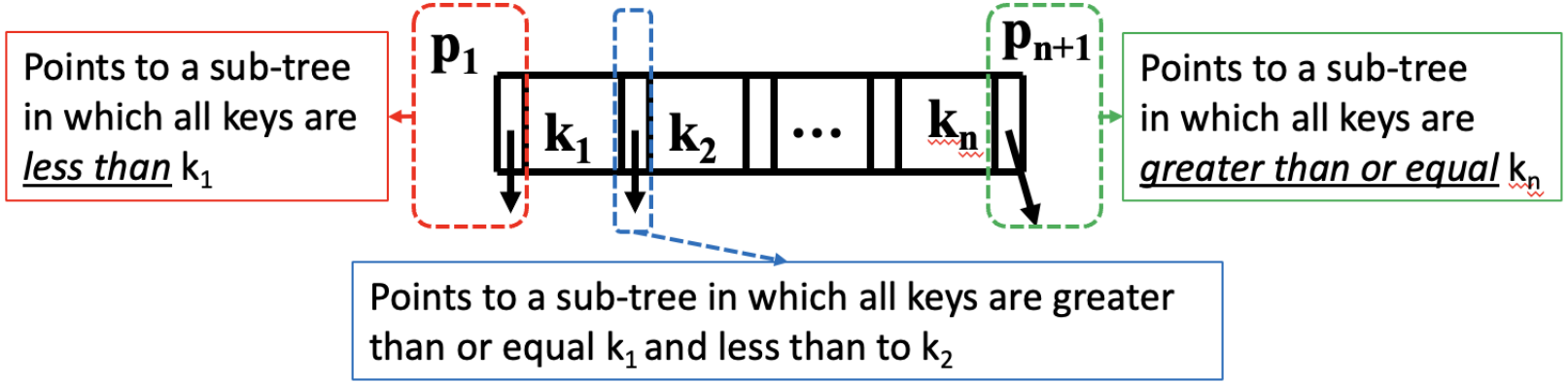
Dynamic Multilevel Indexes Using B+ Trees

Dynamic Trees

- ISAM indices are static
  - Long overflow chains can develop as the file grows, leading to poor performance
- This calls for more flexible, *dynamic* indices that adjust gracefully to insertions and deletions
  - No need to allocate the leaf pages sequentially as in ISAM
- Among the **most successful** dynamic index schemes is the B+ tree

B+ Tree Properties

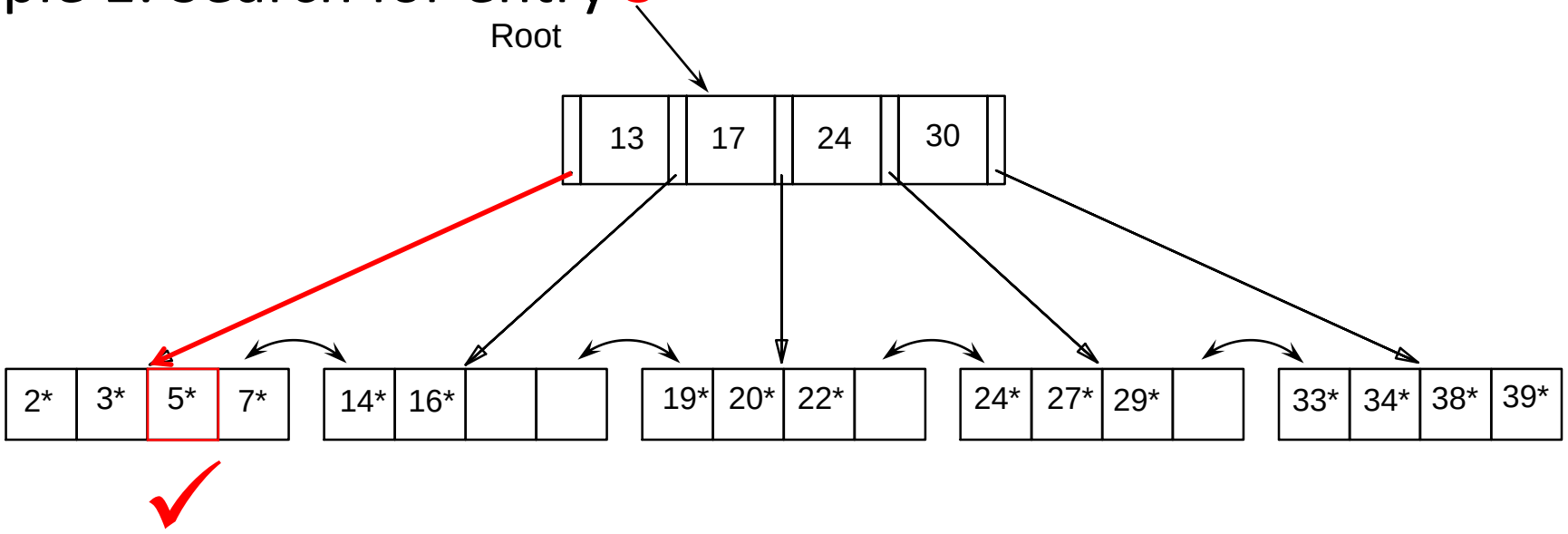
- Each node in a B+ tree of order ***d*** (this is a measure of the capacity of a tree):
  - Has at most ***2d*** keys
  - Has at least ***d*** keys (except the root, which may have just 1 key)
  - All leaves are on the same level
  - Has exactly ***n-1*** keys if the number of pointers is ***n***



B+ Tree: Searching for Entries

- Search begins at root, and key comparisons direct it to a leaf (as in ISAM)

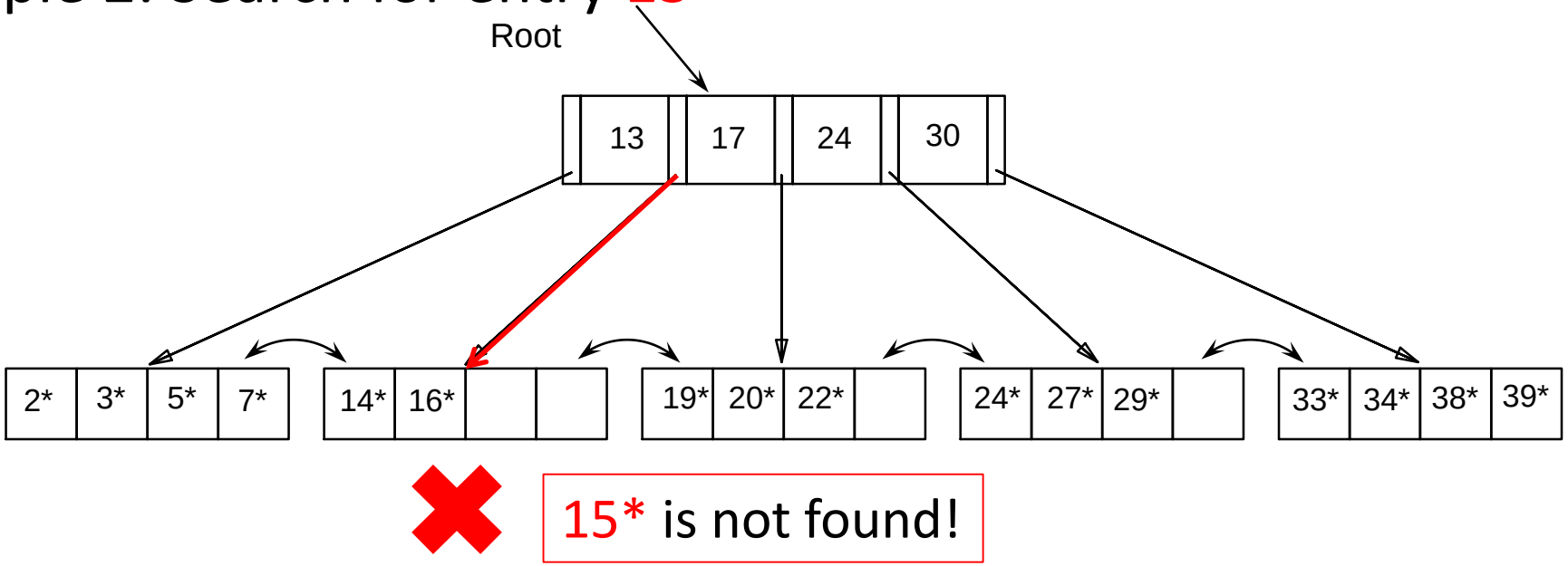
- Example 1: Search for entry 5*



B+ Tree: Searching for Entries

- Search begins at root, and key comparisons direct it to a leaf (as in ISAM)

- Example 2: Search for entry 15*



B+ Trees: Inserting Entries

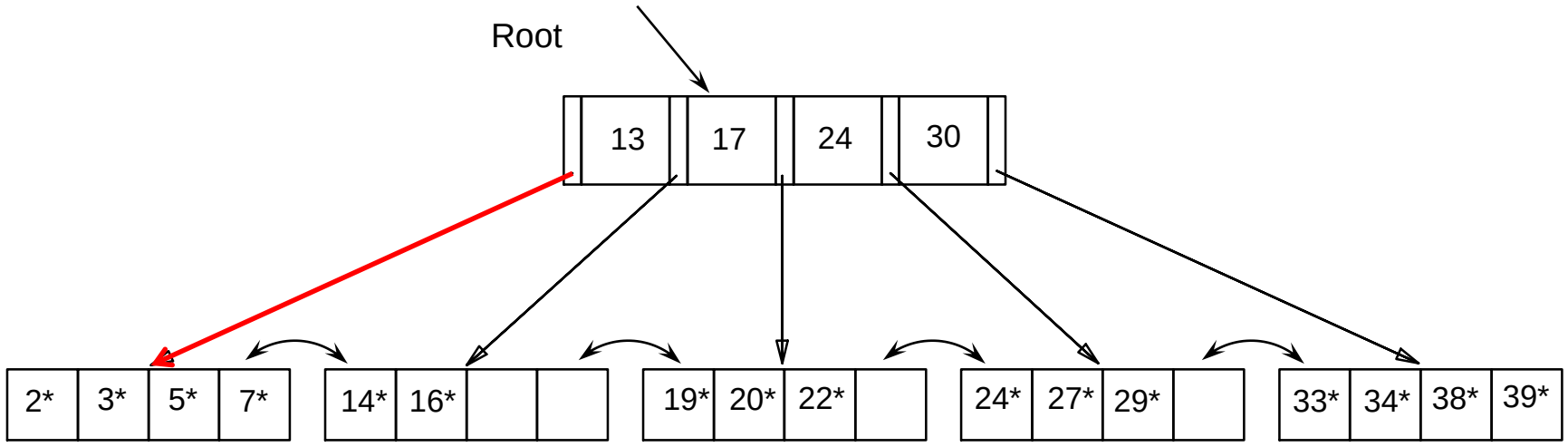
- Find correct leaf ***L***

- Put data entry onto ***L***
  - If ***L*** has enough space, *done*!
  - Else, *split* ***L*** *into* ***L*** *and a new node* $\boldsymbol{L_2}$
    - Re-partition entries *evenly*, *copying up* the middle key

- Parent node may *overflow*
  - *Push up* middle key (splits "grow" trees; a root split increases the height of the tree)

B+ Tree: Examples of Insertions

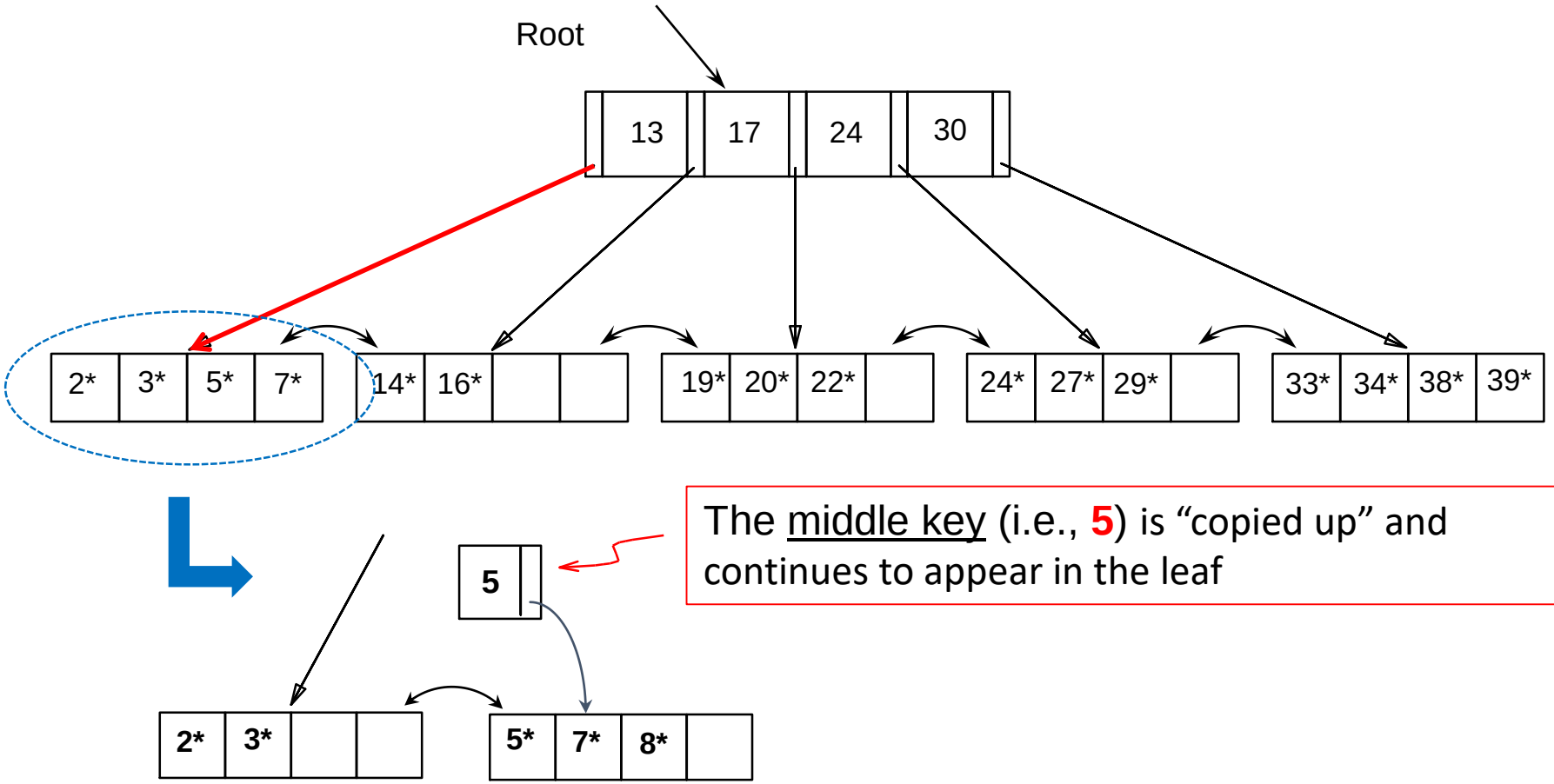
- Insert entry 8*



Leaf is ***full***; hence, split!

B+ Tree: Examples of Insertions

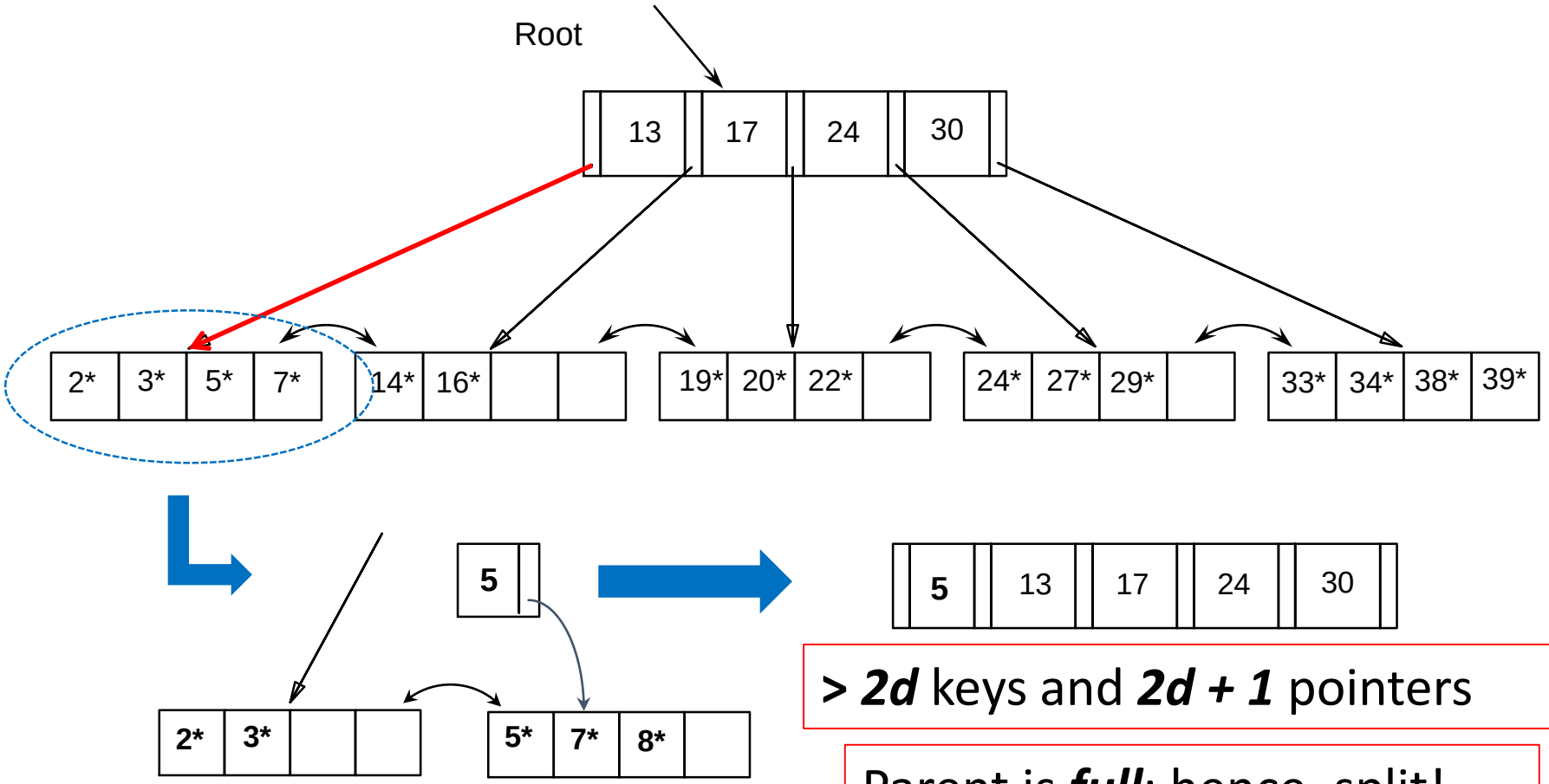
- Insert entry 8*



The <u>middle key</u> (i.e., **5**) is "copied up" and continues to appear in the leaf

B+ Tree: Examples of Insertions

- Insert entry 8*

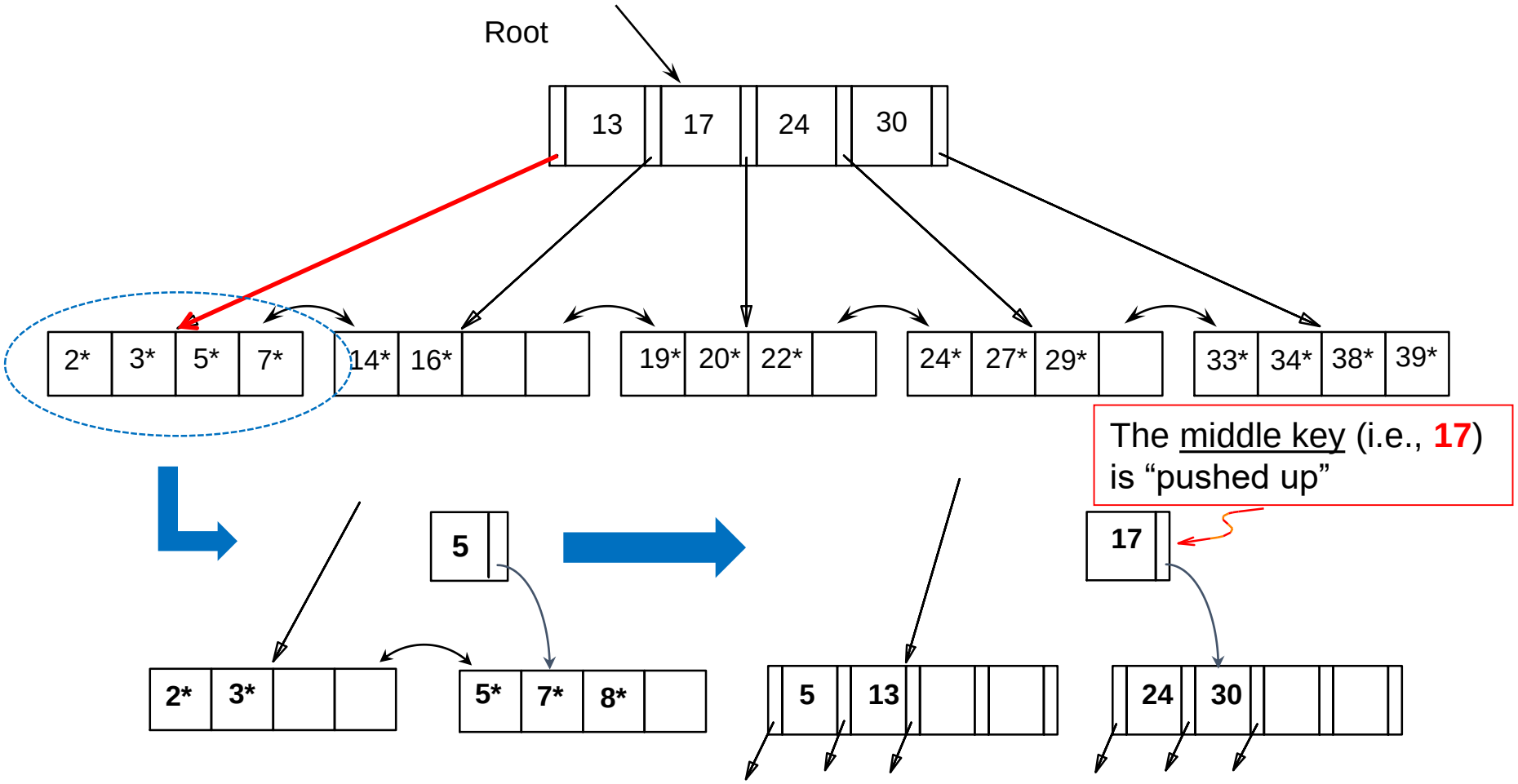


> ***2d*** keys and ***2d + 1*** pointers

Parent is ***full***; hence, split!

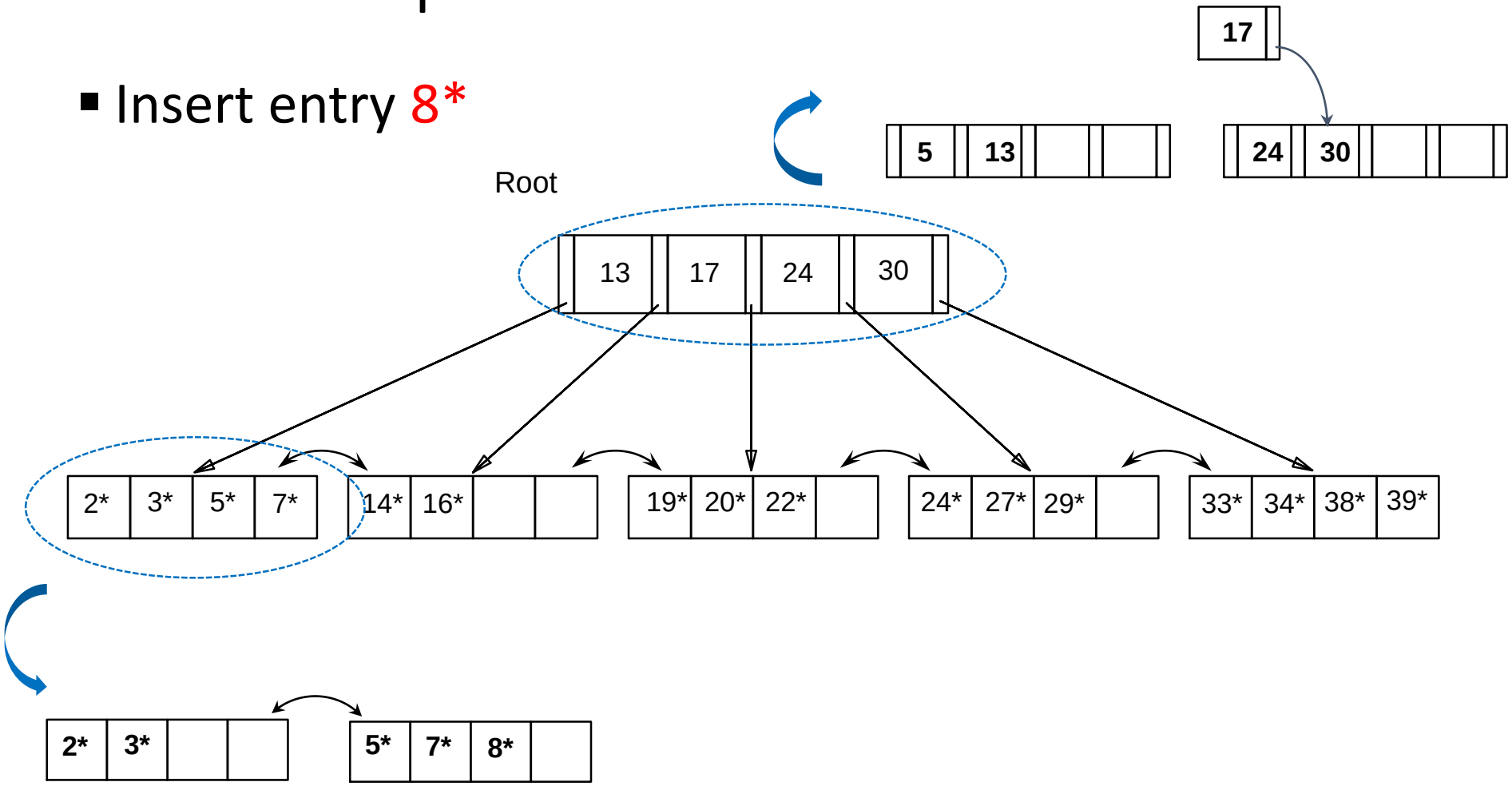
B+ Tree: Examples of Insertions

- Insert entry 8*



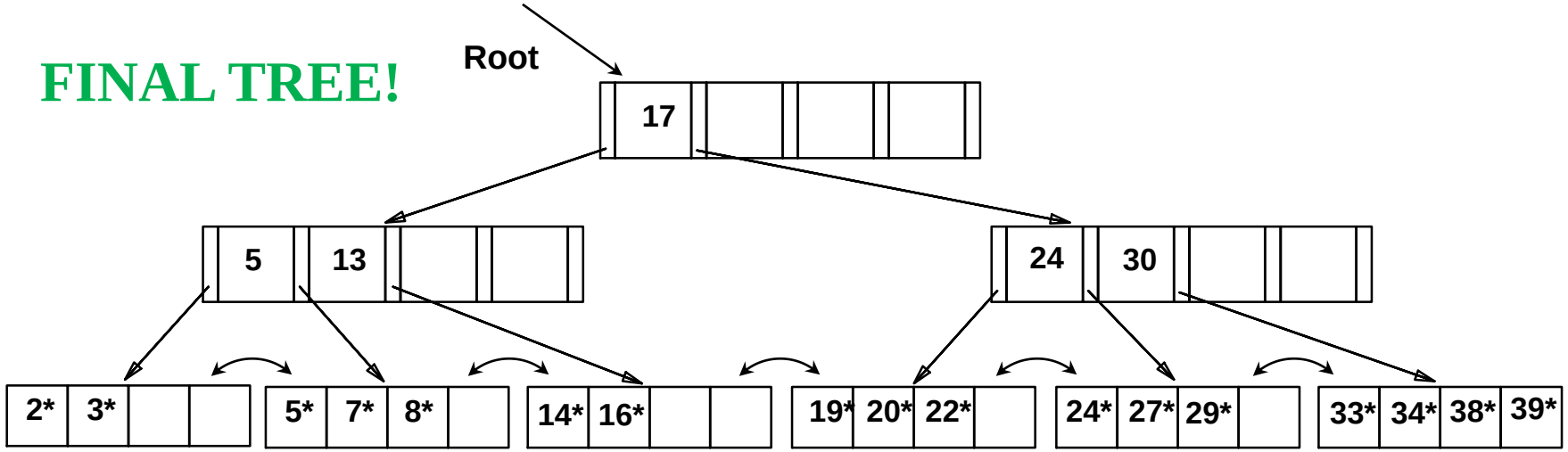
B+ Tree: Examples of Insertions

- Insert entry 8*



B+ Tree: Examples of Insertions

- Insert entry 8*

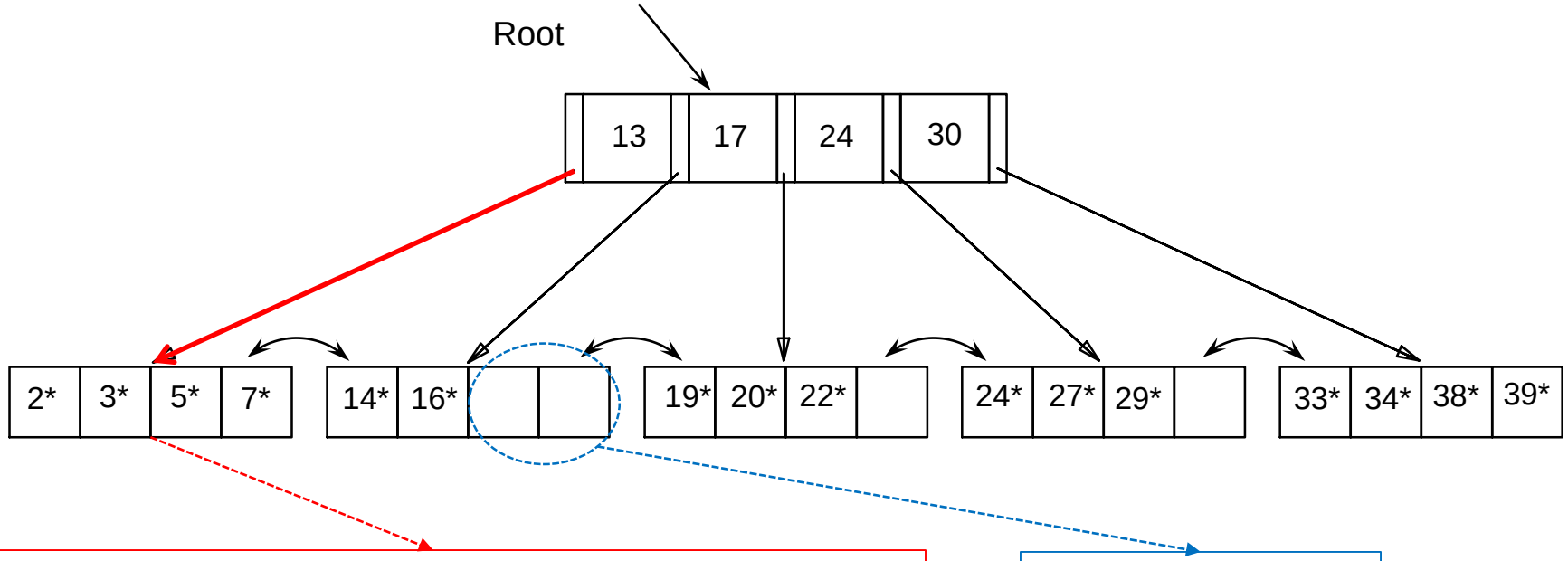


Splitting the root lead to an increase of height by 1!

What about *re-distributing* entries instead of *splitting* nodes?

B+ Tree: Examples of Insertions

- Insert entry 8*

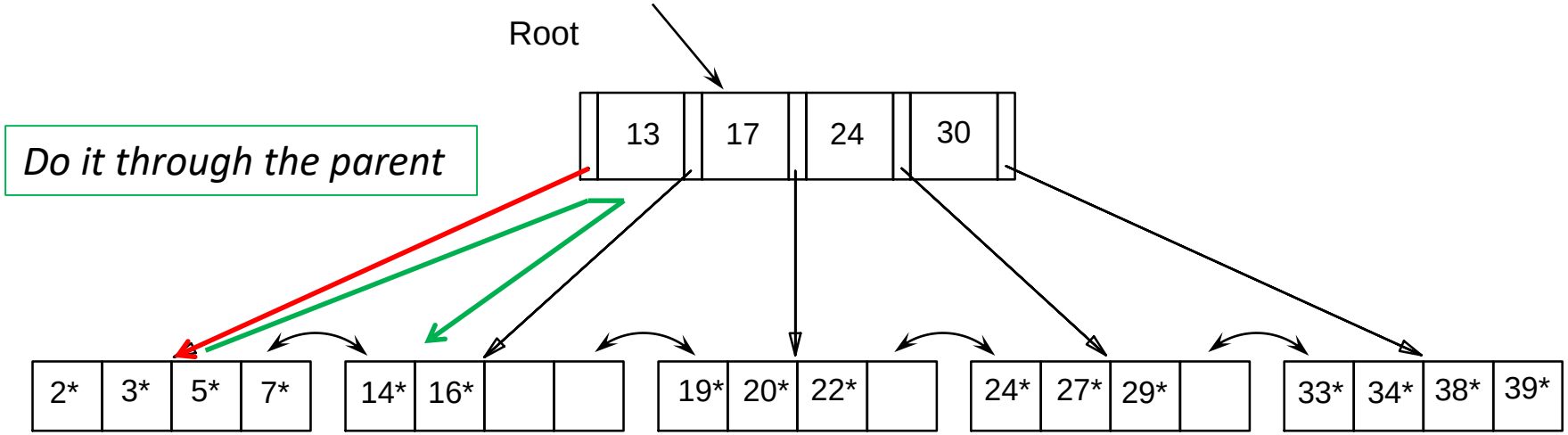


Leaf is ***full***; hence, check the sibling

'Poor Sibling'

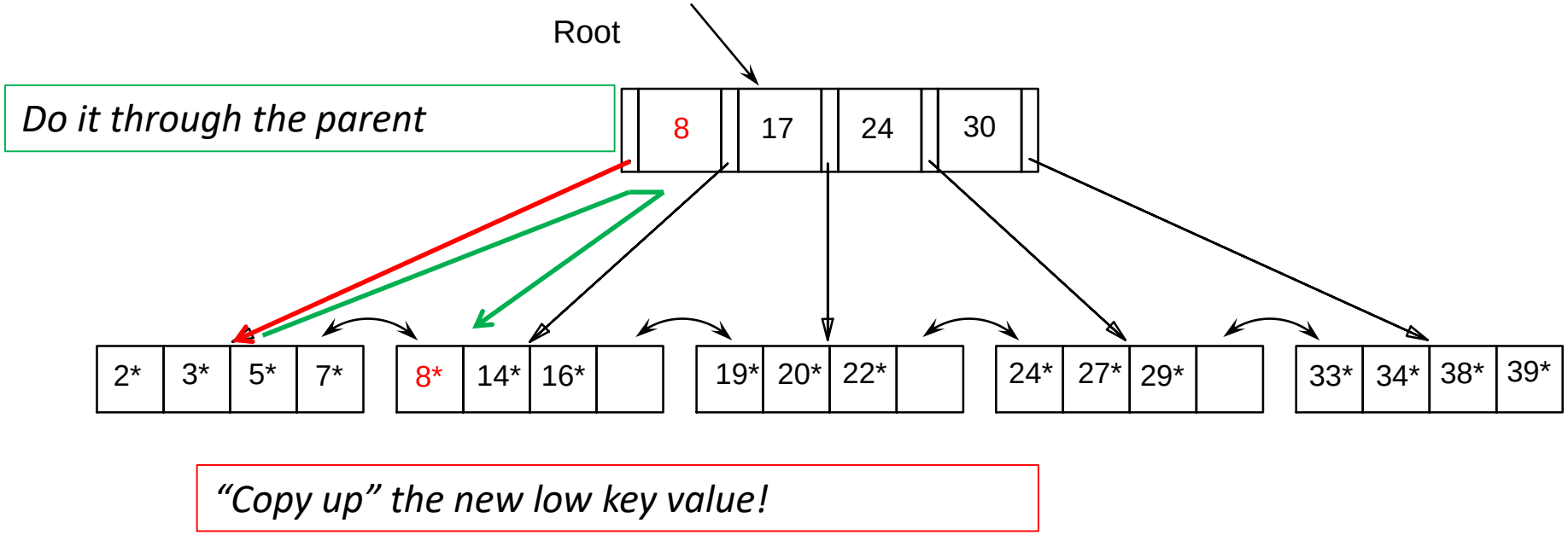
B+ Tree: Examples of Insertions

- Insert entry 8*



B+ Tree: Examples of Insertions

- Insert entry 8*



But, when to *redistribute* and when to *split*?

Splitting vs. Redistributing

- Leaf Nodes
  - Previous and next-neighbor pointers must be updated upon insertions (*if splitting is to be pursued*)
  - Checking whether redistribution is possible does not increase I/O
  - Therefore, if a sibling can spare an entry, re-distribute

- Non-Leaf Nodes
  - Checking whether redistribution is possible *usually* increases I/O
  - Splitting non-leaf nodes typically pays off!

B+ Insertions: Keep in Mind

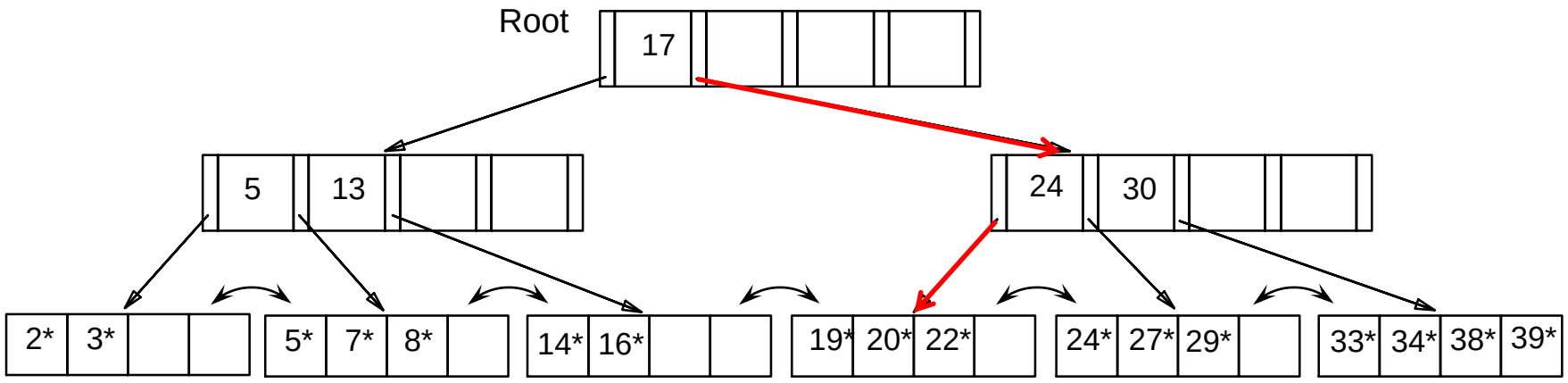
- Every data entry must appear in a leaf node; hence, “copy up” the middle key upon splitting
- When splitting index entries, simply “push up” the middle key
- Apply splitting and/or redistribution on leaf nodes
- Apply only splitting on non-leaf nodes

B+ Trees: Deleting Entries

- Start at root, find leaf ***L*** where entry belongs
- Remove the entry
  - If ***L*** is at least half-full, *done!*
  - If ***L*** *underflows*
    - Try to re-distribute (i.e., borrow from a "rich sibling" and "copy up" its *lowest key*)
    - If re-distribution fails, *merge* ***L*** and a "poor sibling"
      - Update parent
      - And possibly merge, recursively

B+ Tree: Examples of Deletions

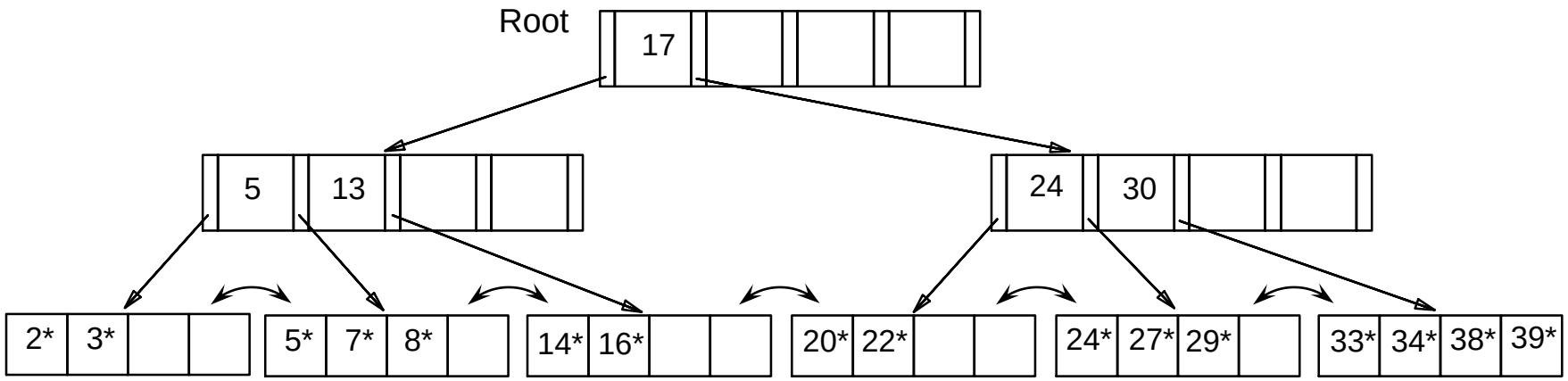
- Delete 19*



Removing 19* does not cause an underflow

B+ Tree: Examples of Deletions

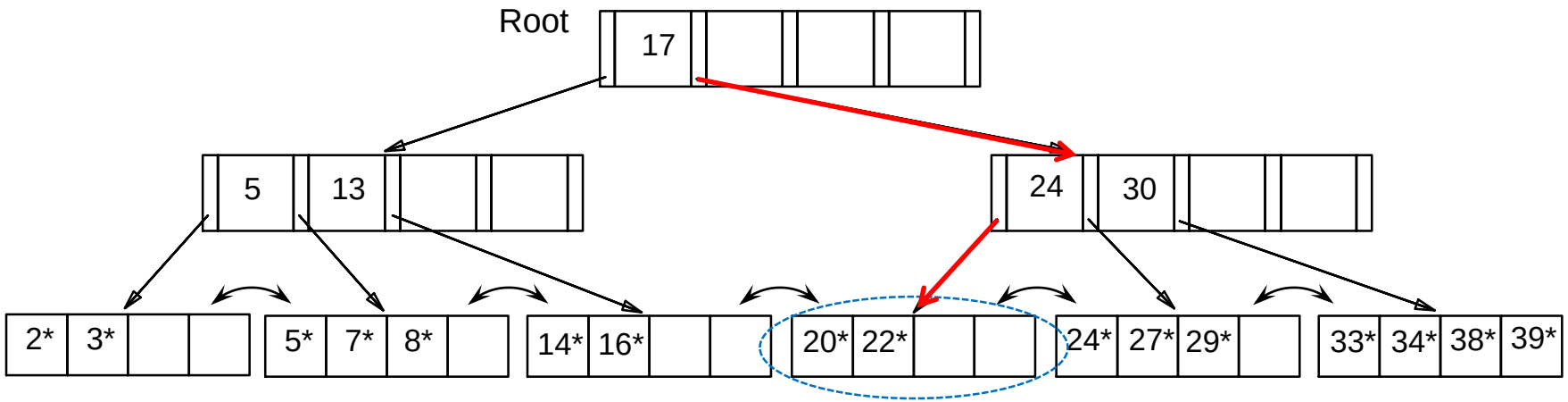
- Delete 19*



FINAL TREE!

B+ Tree: Examples of Deletions

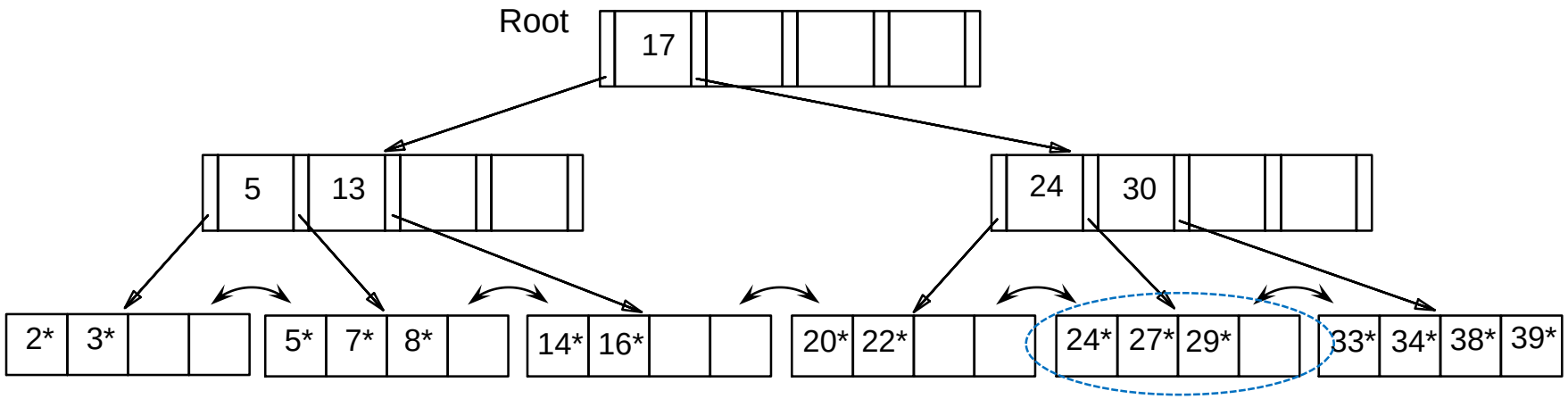
- Delete 20*



Deleting 20* causes an underflow; hence, check a sibling for redistribution

B+ Tree: Examples of Deletions

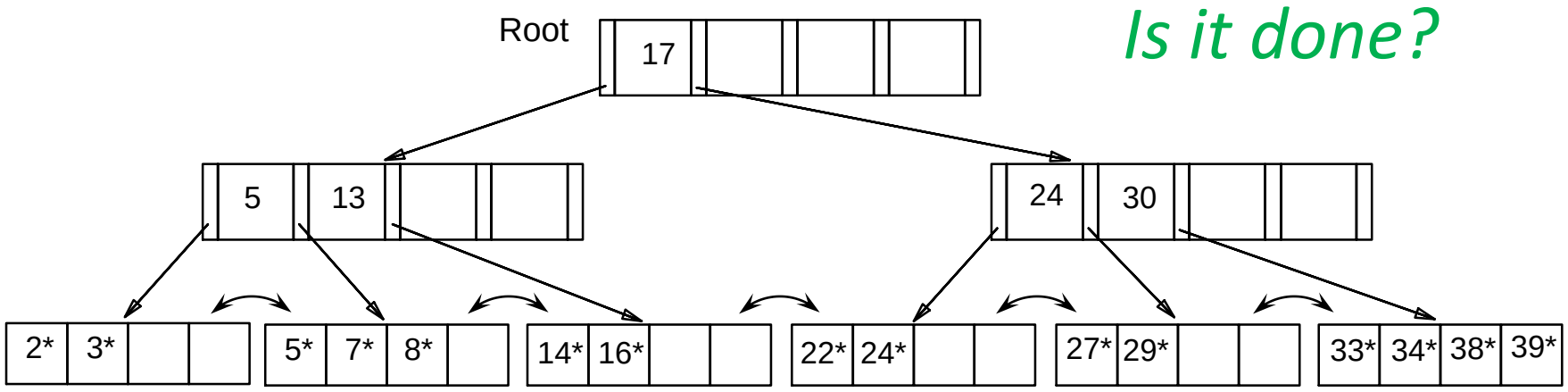
- Delete 20*



The sibling is ‘rich’ (i.e., can lend an entry); hence, remove 20* and redistribute!

B+ Tree: Examples of Deletions

- Delete 20*

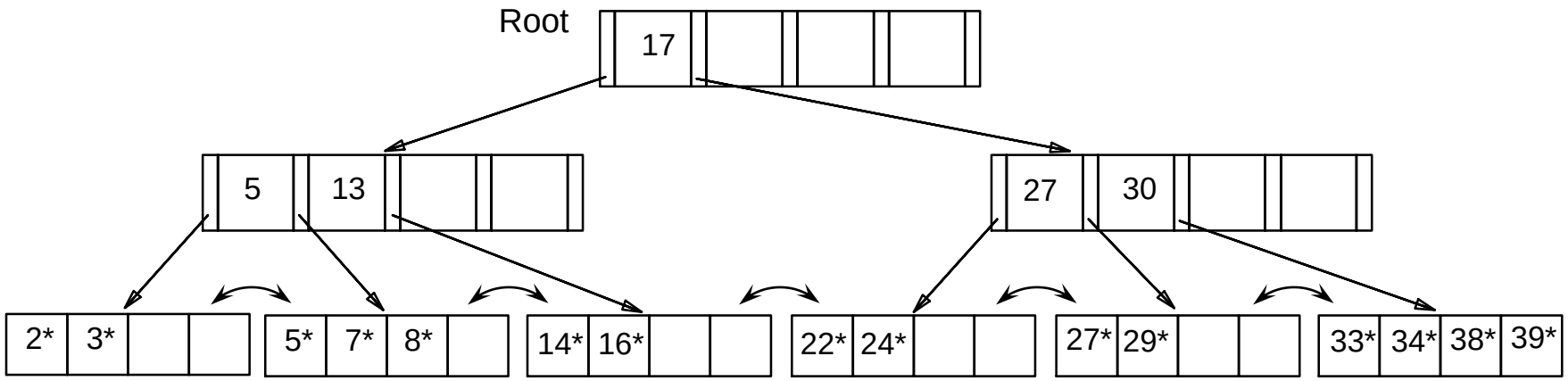


Is it done?

"Copy up" 27*, the lowest value in the leaf from which we borrowed 24*

B+ Tree: Examples of Deletions

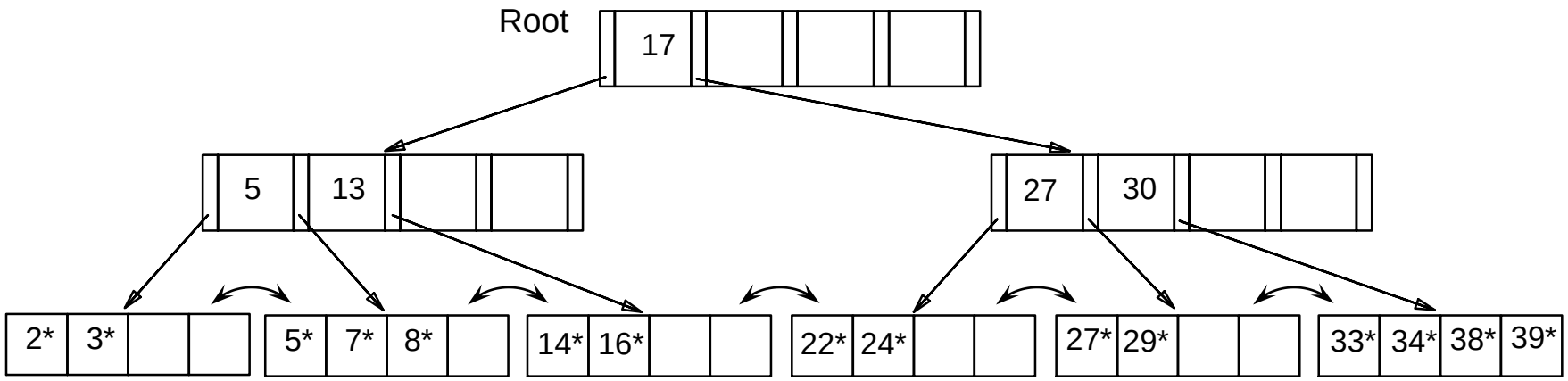
- Delete 20*



"Copy up" 27*, the lowest value in the leaf from which we borrowed 24*

B+ Tree: Examples of Deletions

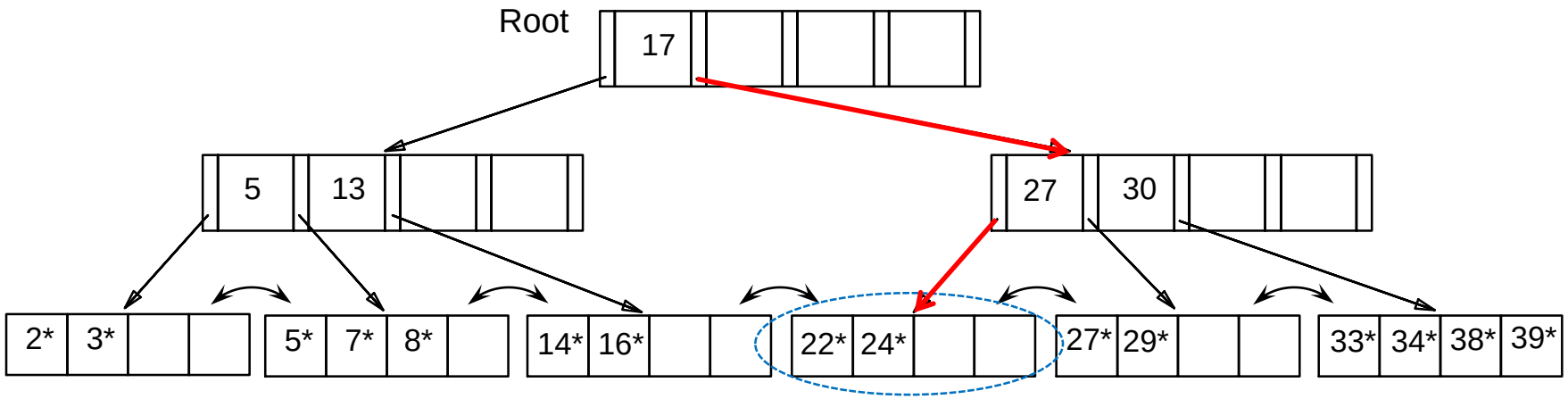
- Delete 20*



FINAL TREE!

B+ Tree: Examples of Deletions

- Delete 24*

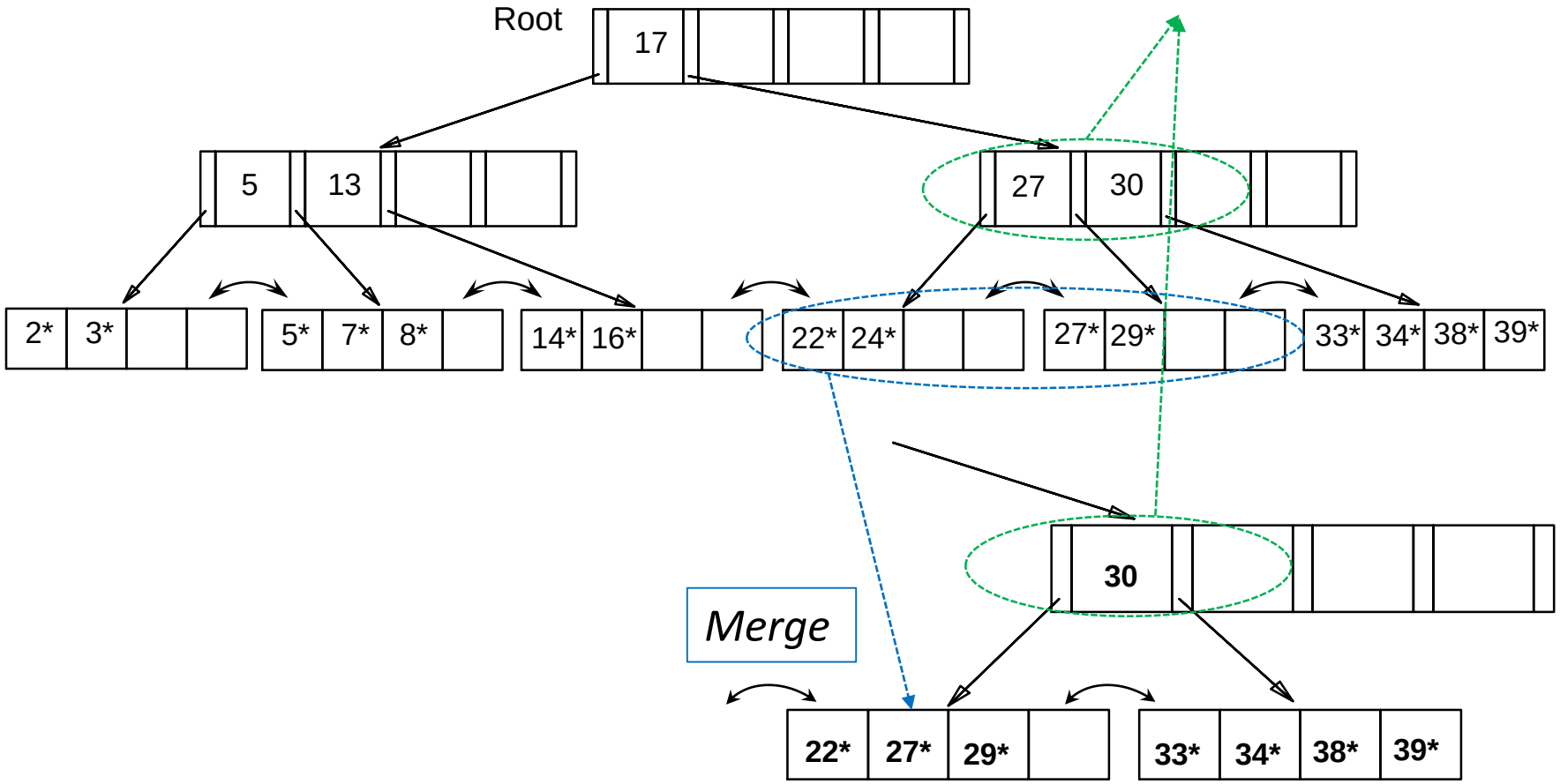


The affected leaf will contain only 1 entry and the sibling cannot lend any entry (i.e., redistribution is not applicable); hence, *merge*!

B+ Tree: Examples of Deletions

- Delete 24*

"Toss" 27 because the page that it was pointing to does not exist anymore!

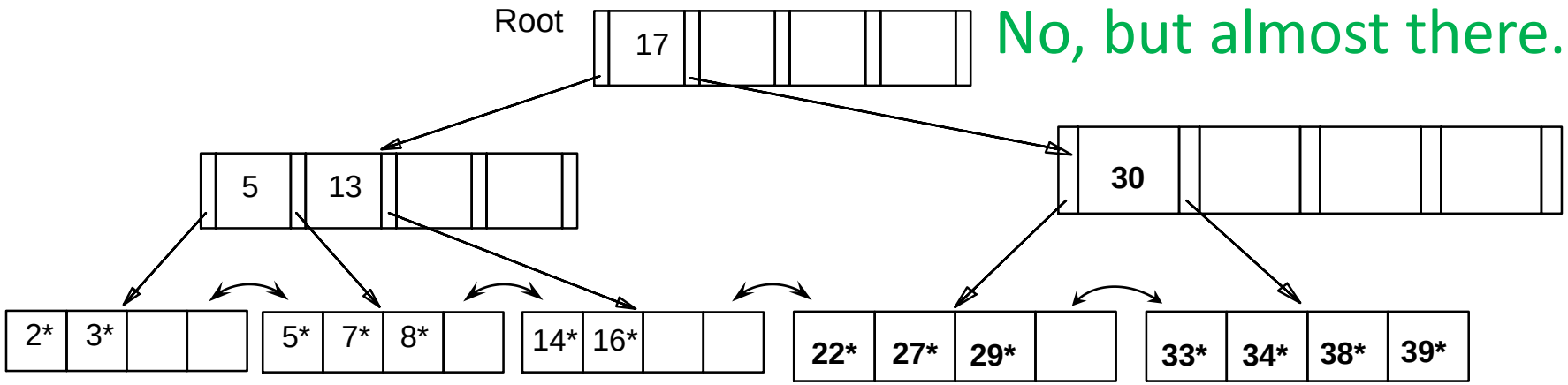


B+ Tree: Examples of Deletions

- Delete 24*

Is it done?

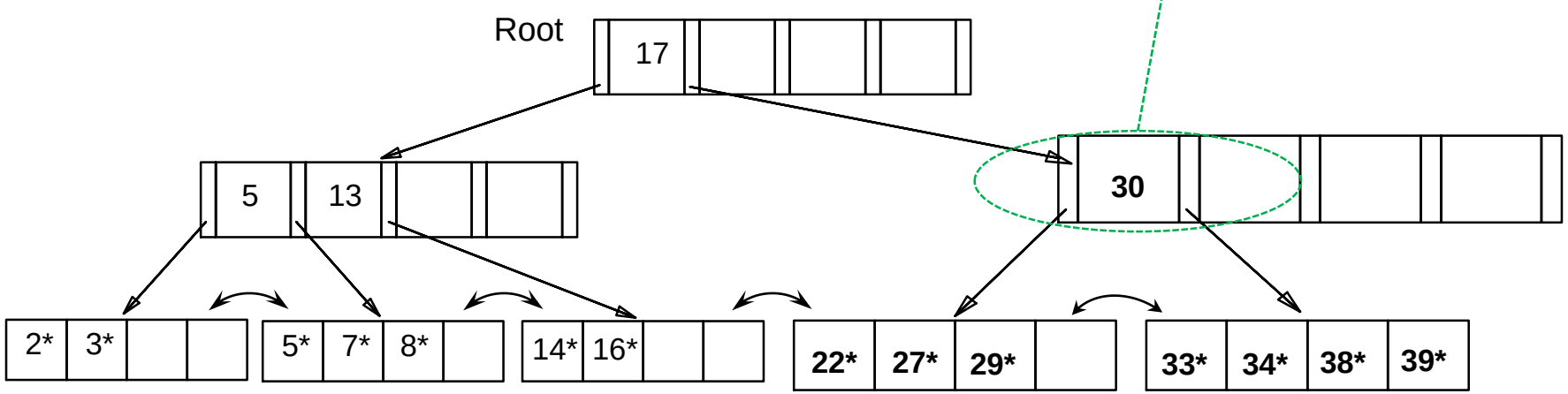
No, but almost there…



B+ Tree: Examples of Deletions

- Delete 24*

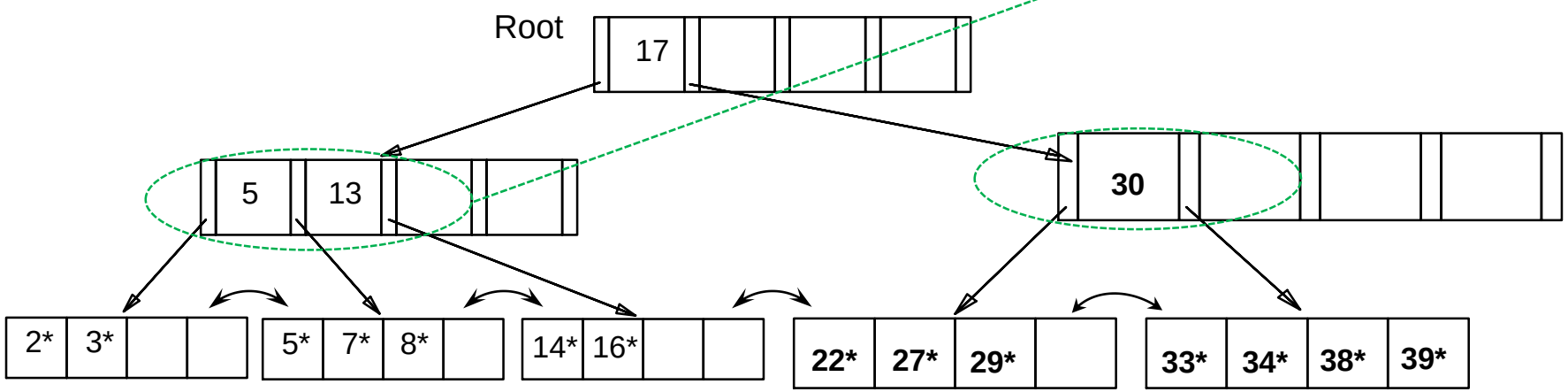
This entails an <u>underflow</u>; hence, we must either redistribute or merge!



B+ Tree: Examples of Deletions

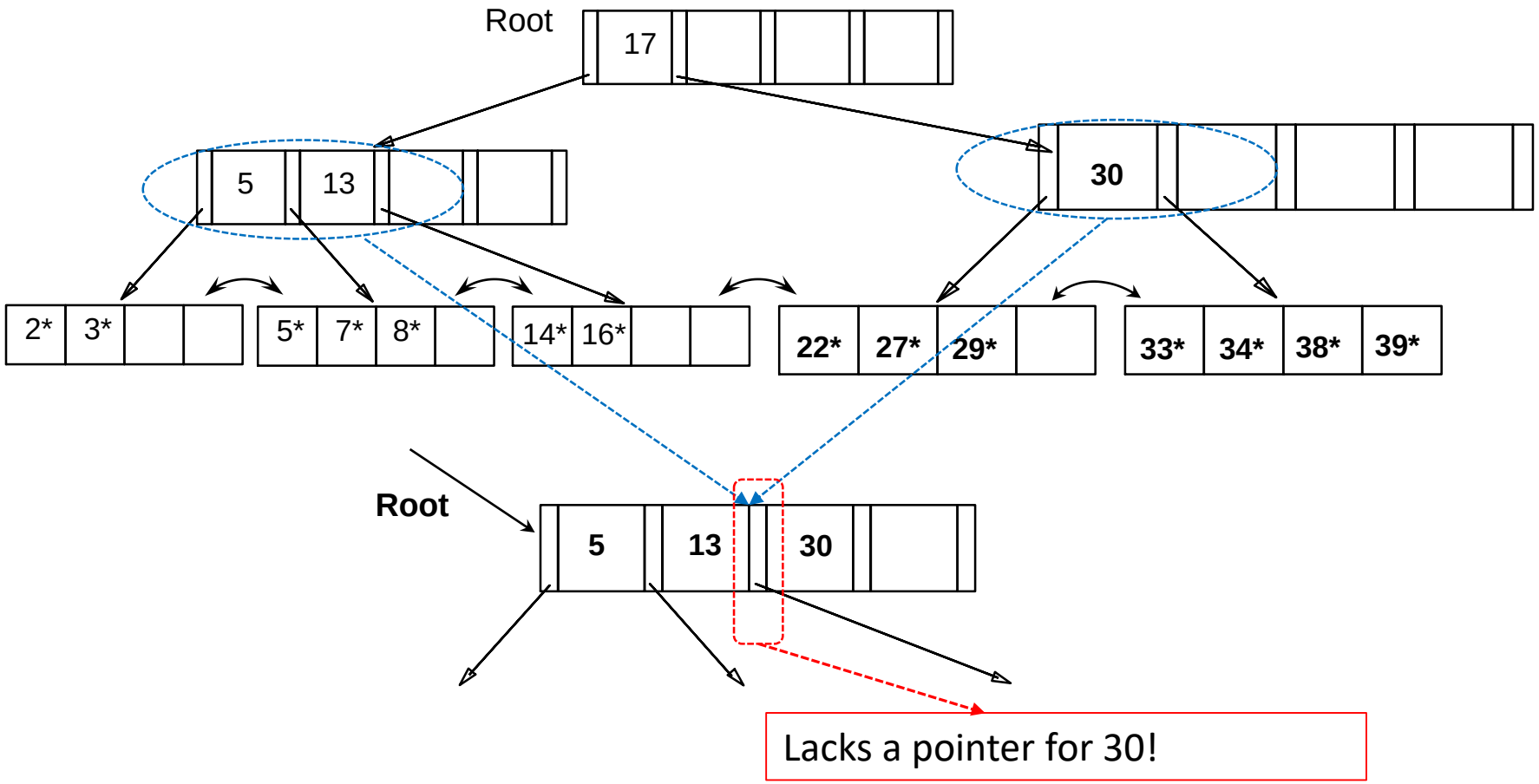
- Delete 24*

The sibling is "poor" (i.e., redistribution is not applicable)*; hence, merge!*



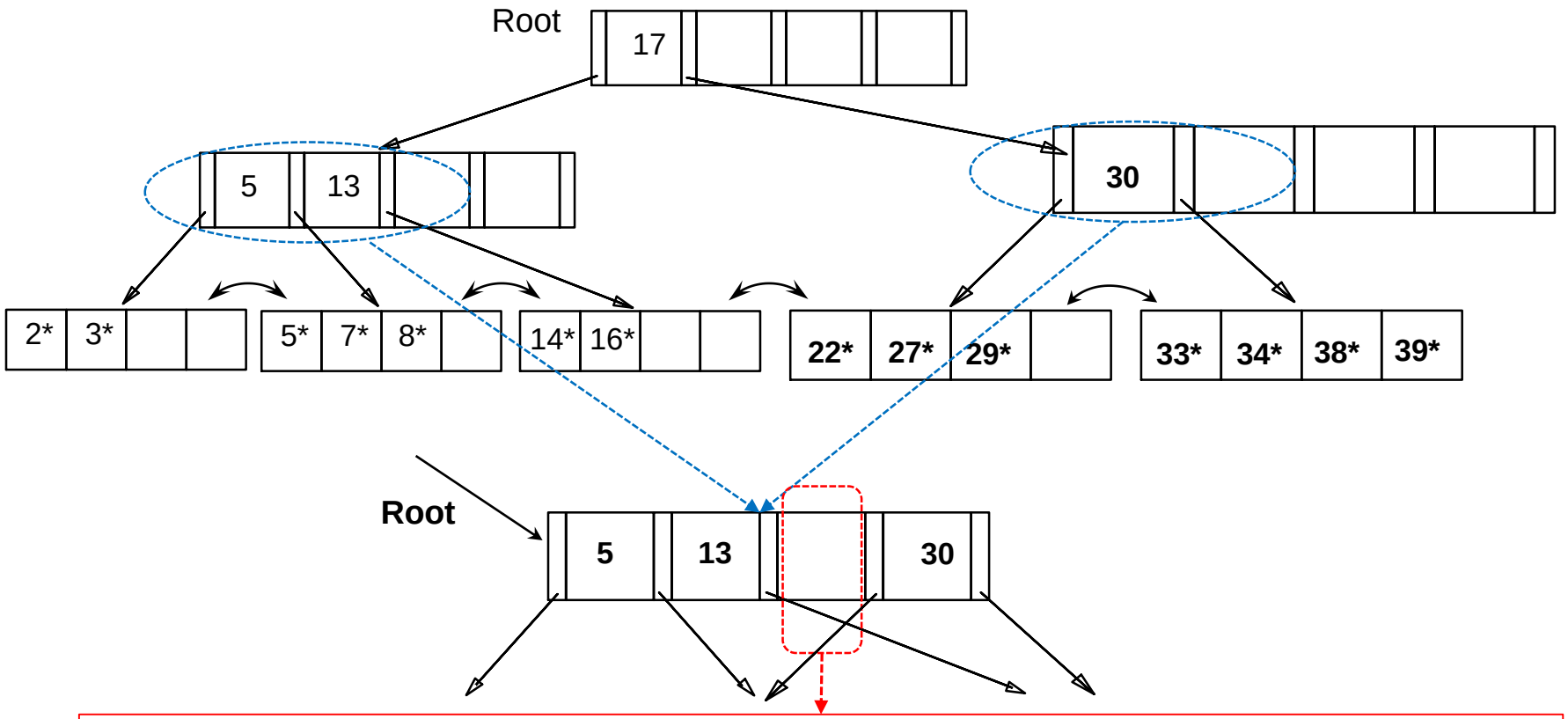
B+ Tree: Examples of Deletions

- Delete 24*



B+ Tree: Examples of Deletions

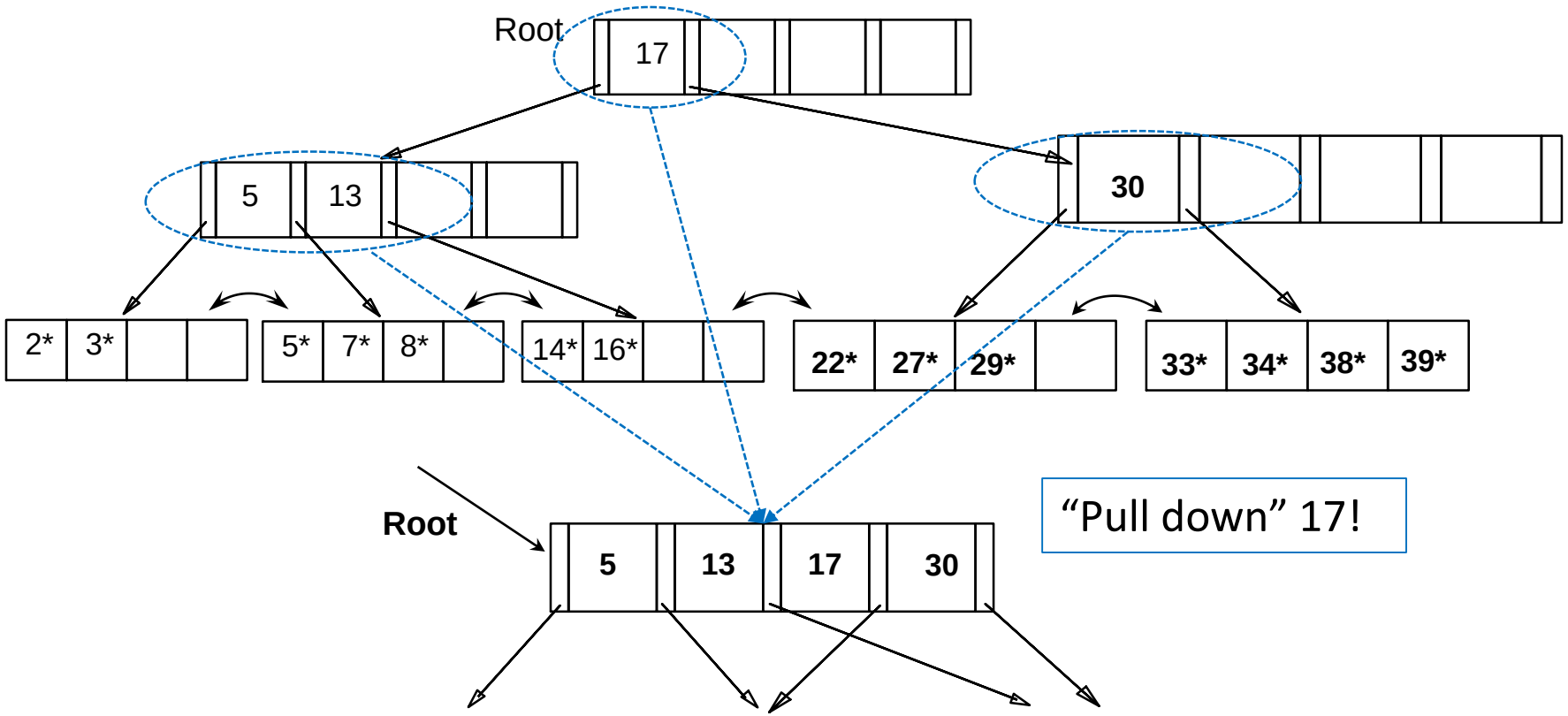
- Delete 24*



Lacks a key value to create a complete index entry!

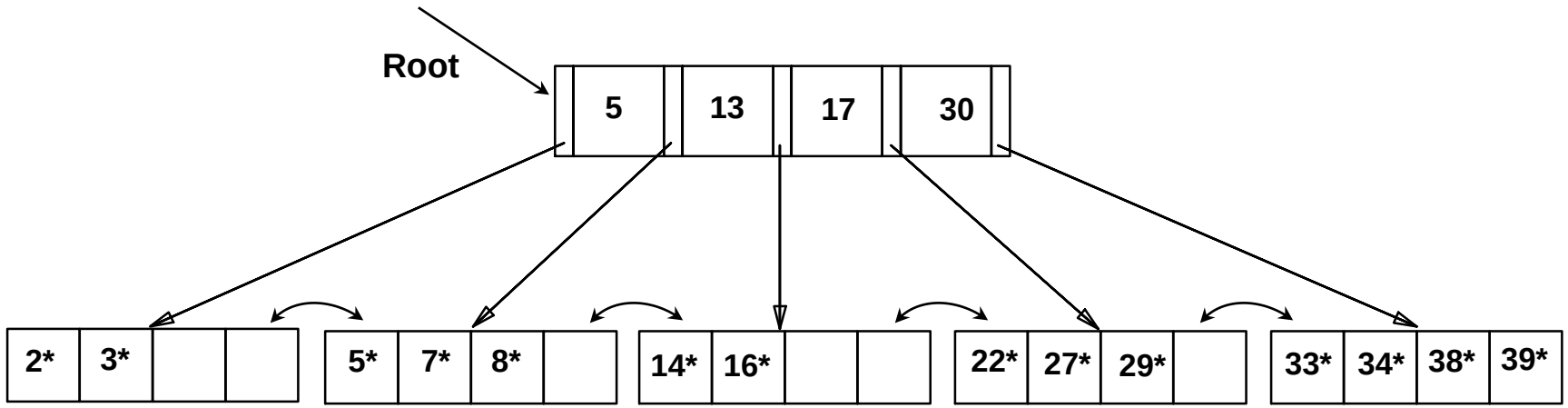
B+ Tree: Examples of Deletions

- Delete 24*



B+ Tree: Examples of Deletions

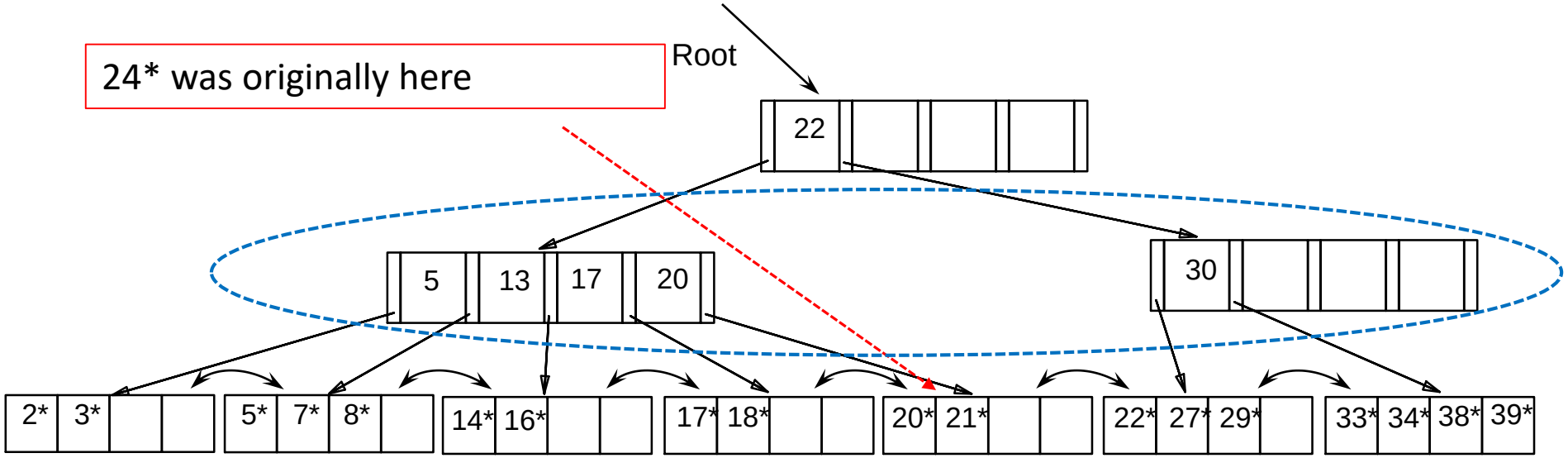
- Delete 24*



FINAL TREE!

B+ Tree: Examples of Deletions

- Delete 24*

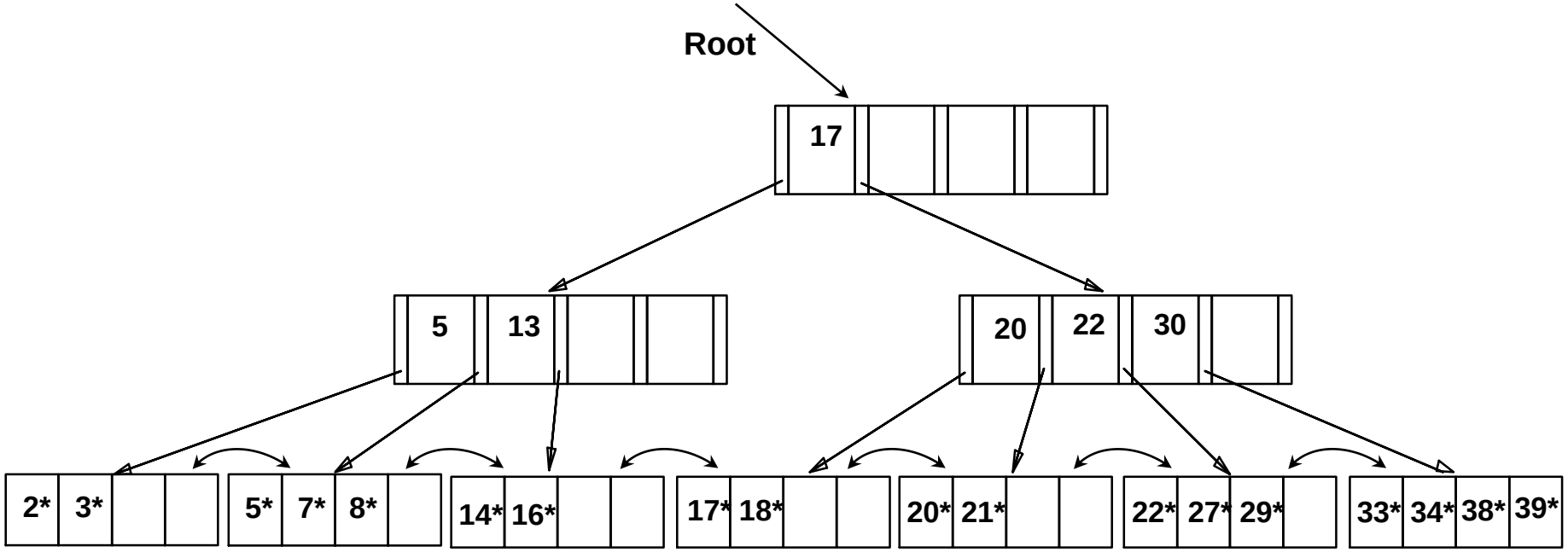


Assume (<u>instead</u>) the above tree *during* deleting 24*

Now we can <u>re-distribute</u> (*instead of merging*) keys!

B+ Tree: Examples of Deletions

- Delete 24*



DONE! It suffices to re-distribute only 20; 17 was redistributed for illustration.